

SWIFT SHIP TRACKER

Salesforce-Based Parcel Management and AI Tracking System

PROJECT DOCUMENTATION

Detailed implementation, configuration, testing, deployment and source-control report.

TEAM LEADER: SURYA P

TEAM MEMBERS:

GOPIKA R

KAVIYA M

PAVITHRA A

SWETHA M

**COLLEGE: GOVERNMENT COLLEGE OF ENGINEERING,
ERODE**

TABLE OF CONTENTS

Document Scope and Implementation Boundary..... 1

1. Introduction..... 2

2. Ideation Phase.....5

3. Requirement Analysis..... 8

4. Technology Stack.....13

5. Project Design..... 14

6. Project Planning.....17

7. Development Environment.....22

8. Data Model Overview..... 25

9. Custom Tabs..... 30

10. Security Model.....31

11. Prompt Builder..... 33

12. Parcel_Details Flow.....35

13. Agentforce AI Working.....39

14. Testing Strategy.....41

15. Salesforce CLI Validation.....42

16. GitHub Integration.....43

17. Source Code Organization.....44

18. Deployment Considerations.....45

19. Project Limitations.....46

20. Advantages.....46

21. Future Scope.....46

22. Conclusion.....47

23. Project Demonstration Sequence.....47

24. Appendix A — CLI Commands.....48

25. Appendix B — Metadata Inventory.....48

26. Appendix C — Screenshot Checklist.....48

27. Appendix D — Screenshot Placement Guide.....49

28. Appendix E — Final Evidence.....49

29. Final Review Checklist.....49

30. Final Submission Notes.....	49
1.1 Project Overview.....	2
1.2 Project Objectives.....	3
2.1 Problem Statement.....	5
2.2 User Personas.....	6
2.3 Brainstorming and Selection.....	7
3.1 Functional Requirements.....	9
3.2 Non-Functional Requirements.....	10
3.3 Data Flow.....	11
3.4 AI Tracking Flow.....	12
5.1 Problem–Solution Fit.....	14
5.2 Proposed Solution.....	15
5.3 Solution Architecture.....	15
6.1 Work Breakdown.....	17
7.1 Salesforce DX Structure.....	17
7.2 Salesforce Org Authentication.....	22
7.3 Org Metadata Verification.....	23
8.1 Parcel__c Object.....	25
8.2 Parcel Fields.....	26
8.3 Delivery__c Object.....	26
8.4 Delivery Fields.....	27
8.5 Sender__c Object.....	28
8.6 Receiver__c Object.....	29
8.7 Object Relationships.....	29
9.1 List Views.....	30
10.1 Swift_Ship Permission Set.....	32
10.2 SwiftShip Tracker Permissions.....	32
11.1 Prompt Resources.....	33
11.2 Prompt Response Format.....	34
12.1 Flow Variables.....	35
12.2 Get Parcel Records.....	36
12.3 Parcel Updates Action.....	37
12.4 Assignment Outputs.....	37
12.5 Flow Sequence.....	38
13.1 End-to-End AI Lifecycle.....	39
13.2 Why the AI Agent Matters.....	40
14.1 Functional Test Cases.....	41
14.2 Flow Testing.....	42
14.3 AI Testing.....	42
15.1 Successful Dry Run.....	42
15.2 Metadata Validation Summary.....	42
16.1 Git Status.....	43
16.2 Git Workflow.....	43
17.1 Important Files.....	44

18.1 Troubleshooting History.....45

20.1 Disadvantages.....46

LIST OF FIGURES

Figure 0.1: Project Structure / Scope.....	1
Figure 1.1: Salesforce Project Home.....	2
Figure 1.2: Object Overview.....	3
Figure 1.3: Objectives.....	4
Figure 2.1: Ideation.....	5
Figure 2.2: Problem Statement.....	6
Figure 2.3: User Persona.....	7
Figure 2.4: Brainstorming.....	8
Figure 3.1: Requirements Overview.....	9
Figure 3.2: Requirements Table.....	10
Figure 3.3: NFR Table.....	11
Figure 3.4: Data Flow Diagram.....	12
Figure 3.5: AI Flow.....	12
Figure 4.1: Technology Stack.....	13
Figure 5.1: Architecture.....	14
Figure 5.2: Problem-Solution.....	15
Figure 5.3: Proposed Solution.....	15
Figure 5.4: Architecture Diagram.....	16
Figure 6.1: Planning.....	17
Figure 6.2: Work Breakdown.....	18
Figure 7.1: Terminal Project Directory.....	22
Figure 7.2: VS Code Explorer.....	22
Figure 7.3: SF Org Display.....	23
Figure 7.4: Metadata Listing.....	24
Figure 8.1: Object Manager.....	25
Figure 8.2: Parcel Object.....	25
Figure 8.3: Parcel Fields.....	26
Figure 8.4: Delivery Object.....	27
Figure 8.5: Delivery Fields.....	28
Figure 8.6: Sender Object.....	28
Figure 8.7: Receiver Object.....	29
Figure 8.8: Relationship.....	29
Figure 9.1: Custom Tabs.....	30
Figure 9.2: List Views.....	30
Figure 10.1: Permission Sets.....	31
Figure 10.2: Swift_Ship.....	32
Figure 10.3: SwiftShip Tracker Permission.....	32
Figure 11.1: Prompt Builder.....	33
Figure 11.2: Prompt Resource Mapping.....	33
Figure 11.3: Prompt Preview.....	34
Figure 12.1: Flow Canvas.....	35
Figure 12.2: Flow Variables.....	36
Figure 12.3: Get Records.....	36
Figure 12.4: Flow Action.....	37
Figure 12.5: Assignment.....	37

Figure 12.6: Full Flow..... 38

Figure 13.1: Agentforce Builder..... 39

Figure 13.2: AI Lifecycle.....40

Figure 13.3: Agentforce Conversation.....40

Figure 14.1: Testing Overview.....41

Figure 16.1: GitHub Repository.....43

Figure 17.1: VS Code Metadata Tree.....44

Figure 18.1: Deployment Troubleshooting.....45

LIST OF TABLES

Table 3.1: Functional Requirements..... 9

Table 3.2: Non-Functional Requirements.....10

Table 4.1: Technology Stack..... 13

Table 5.1: Problem–Solution Fit.....14

Table 6.1: Work Breakdown.....17

Table 6.6: Product Backlog, Sprint Schedule and Estimation.....19

Table 6.7: Sprint Tracker..... 21

Table 8.1: Parcel Fields.....26

Table 8.2: Delivery Fields.....27

Table 12.1: Flow Variables.....35

Table 14.1: Functional Test Cases..... 41

Table 15.1: Metadata Validation Summary.....42

Table 17.1: Important Files.....44

Table 18.1: Troubleshooting History.....45

Table 25.1: Appendix B — Metadata Inventory.....48

DOCUMENT SCOPE AND IMPLEMENTATION BOUNDARY

This report is based on the actual SwiftShip Tracker Salesforce DX project and target Salesforce Developer Org.

The implemented solution contains four custom objects, custom fields and relationships, custom tabs, list views, permission sets, the Parcel_Details Flow, Prompt Builder/Agentforce integration, Salesforce CLI validation and GitHub source control.

Features such as Apex, Reports, Dashboards, Experience Cloud, external logistics integration and production deployment are treated as future scope unless they are later added.

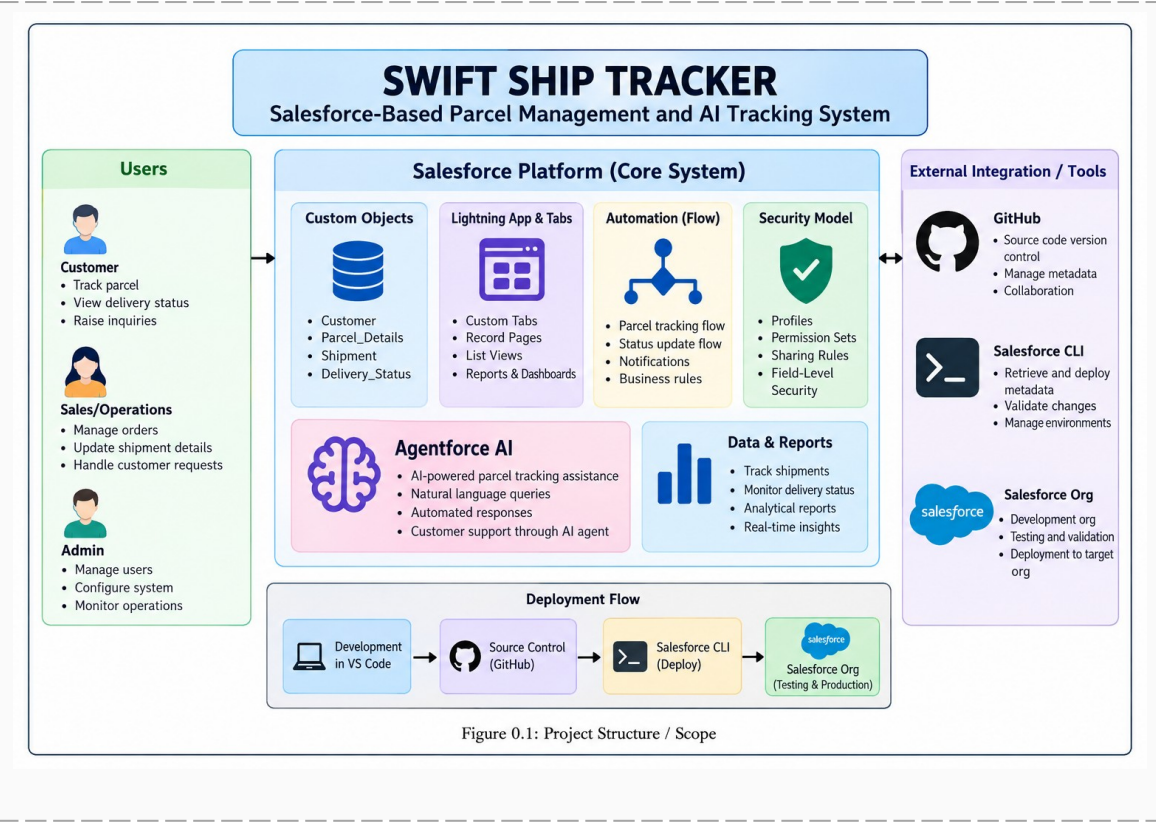


Figure 0.1: Project Structure / Scope

1. INTRODUCTION

SwiftShip Tracker is a Salesforce application for organizing parcel, sender, receiver and delivery information in a centralized platform.

The project demonstrates Salesforce configuration, metadata development, Flow automation, Prompt Builder, Agentforce, permissions, CLI operations and GitHub version control.

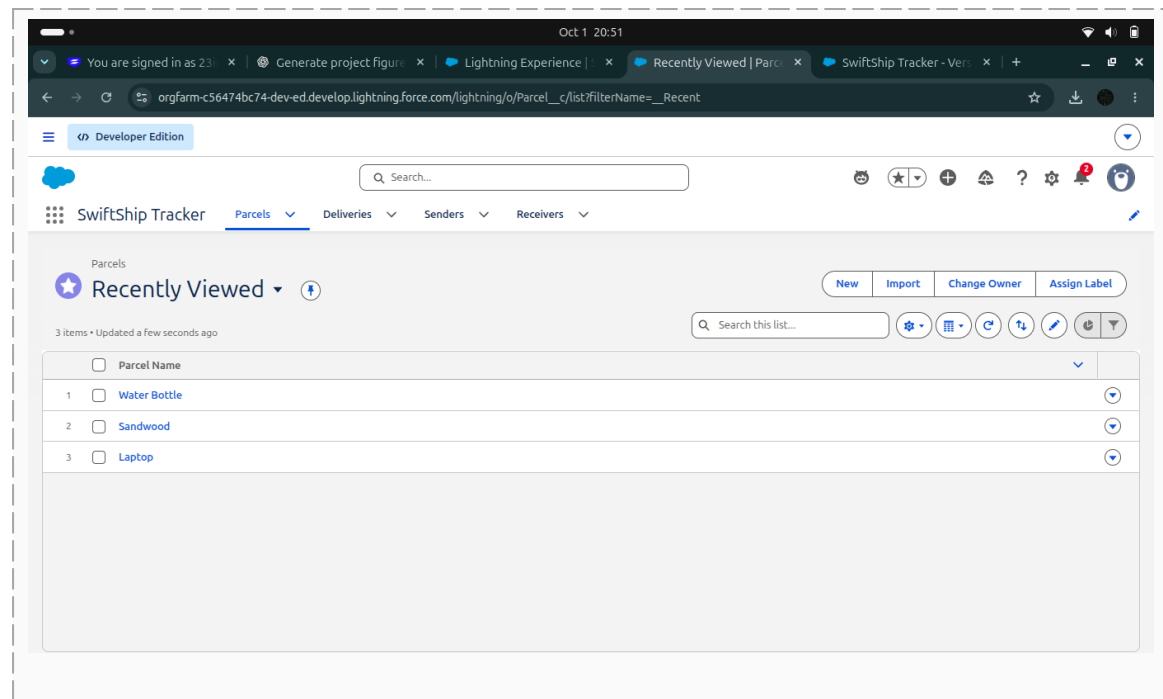


Figure 1.2: Salesforce Project Home

1.1 Project Overview

Parcel__c is the central shipment object. Delivery__c stores delivery information. Sender__c and Receiver__c store the parties involved in a shipment. The Parcel_Details Flow connects parcel data retrieval with the Prompt Builder action so that an AI-driven response can be produced from stored Salesforce data.

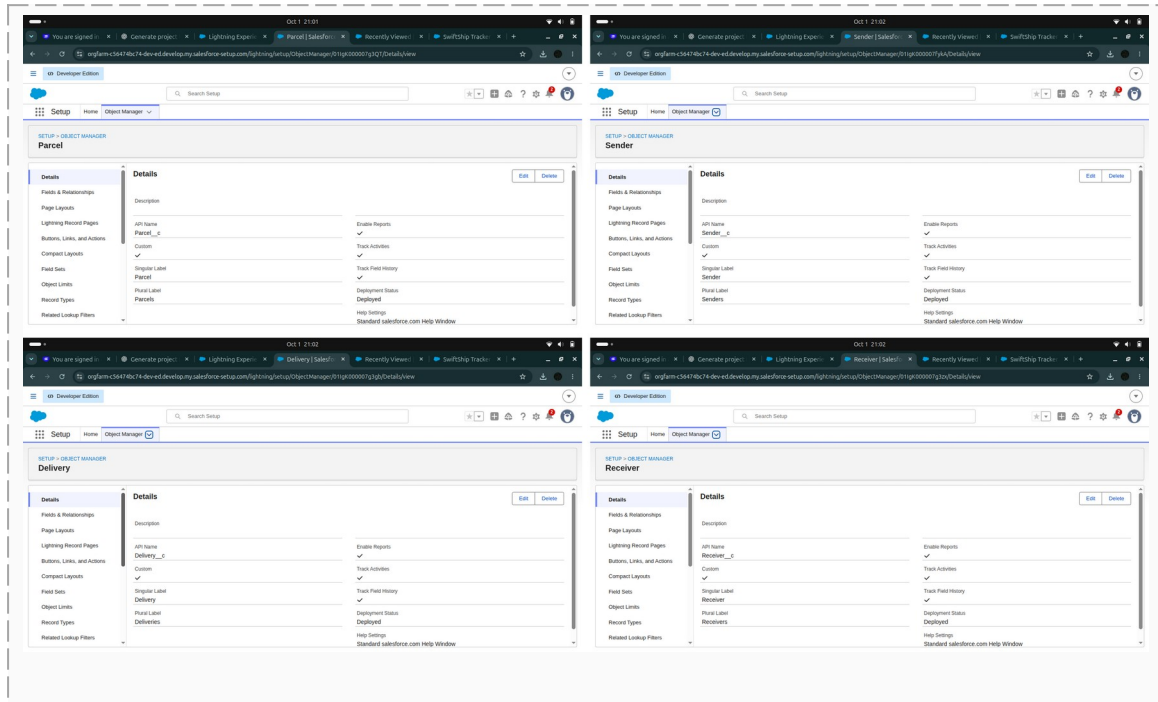


Figure 1.3: Object Overview

1.2 Project Objectives

- Centralize parcel information in Salesforce.
- Maintain sender and receiver information separately.
- Maintain delivery-specific information.
- Retrieve a parcel using Parcel_ID__c.
- Generate a structured AI tracking response.
- Control access using permission sets.
- Maintain Salesforce DX source code.
- Use GitHub for version control.

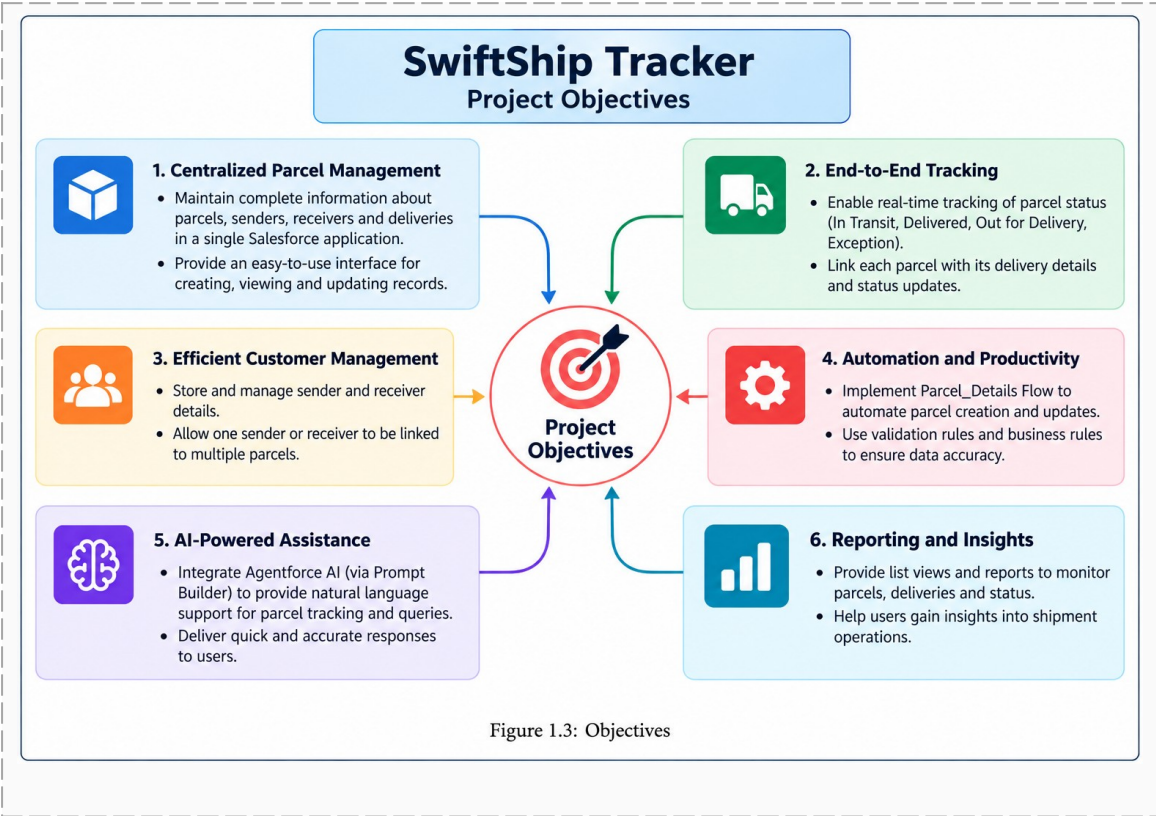


Figure 1.4: Objectives

2. IDEATION PHASE

The reference project identifies parcel booking and tracking as activities that benefit from centralized information and automation.

For this implementation, those ideas were translated into a Salesforce data model and an AI-assisted parcel retrieval workflow.

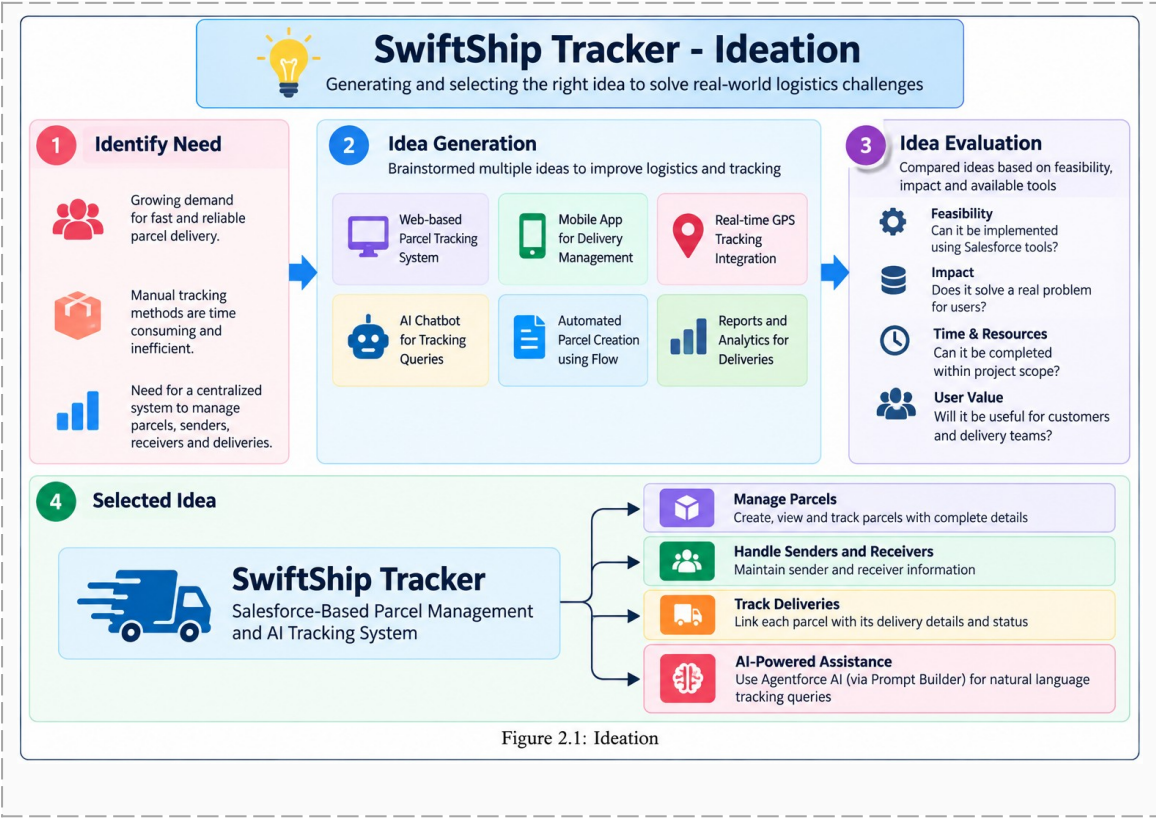


Figure 2.5: Ideation

2.1 Problem Statement

Users need a simple method to find parcel information without navigating through multiple unrelated records. Operations users need structured parcel and delivery data, while an AI experience needs a reliable backend action that retrieves real Salesforce data.

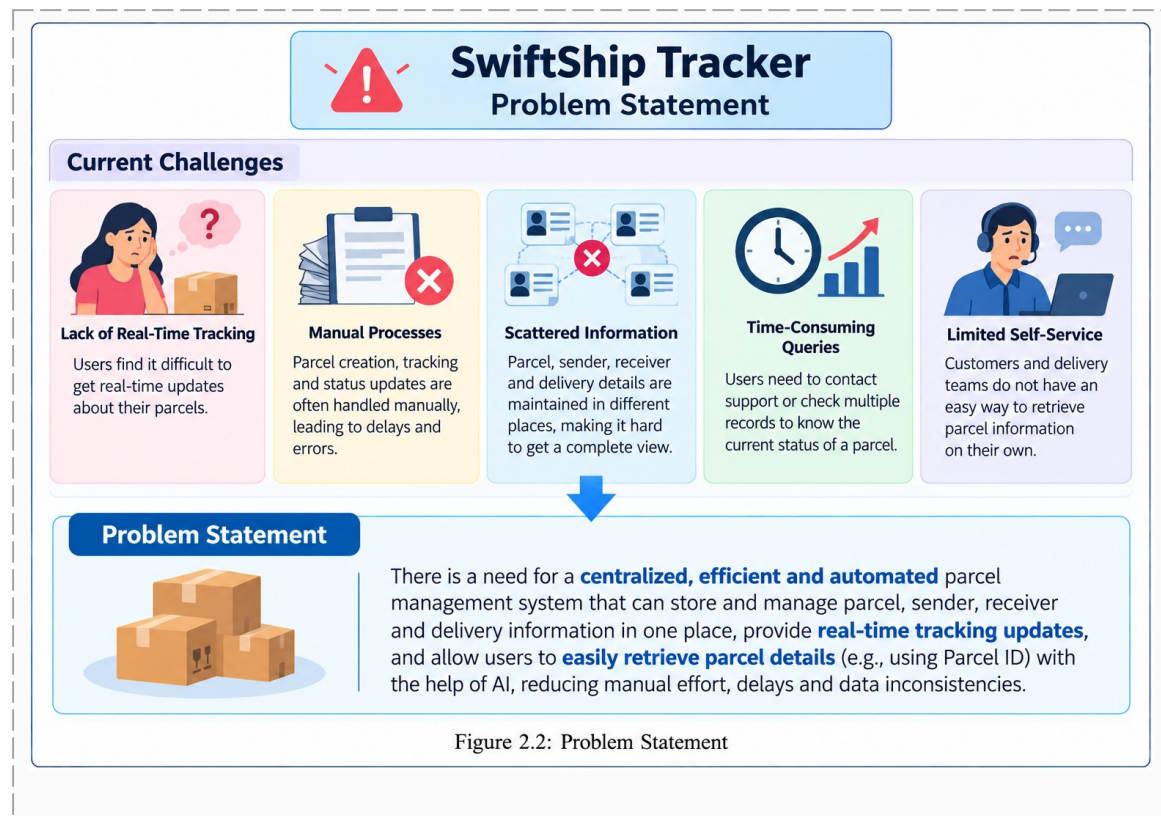


Figure 2.6: Problem Statement

2.2 User Personas

- Customer / Sender — wants convenient access to parcel information.
- Receiver — needs shipment-related information.
- Operations User — manages parcel and delivery records.
- AI / Agent User — invokes the parcel retrieval action.
- Administrator — controls metadata and permissions.

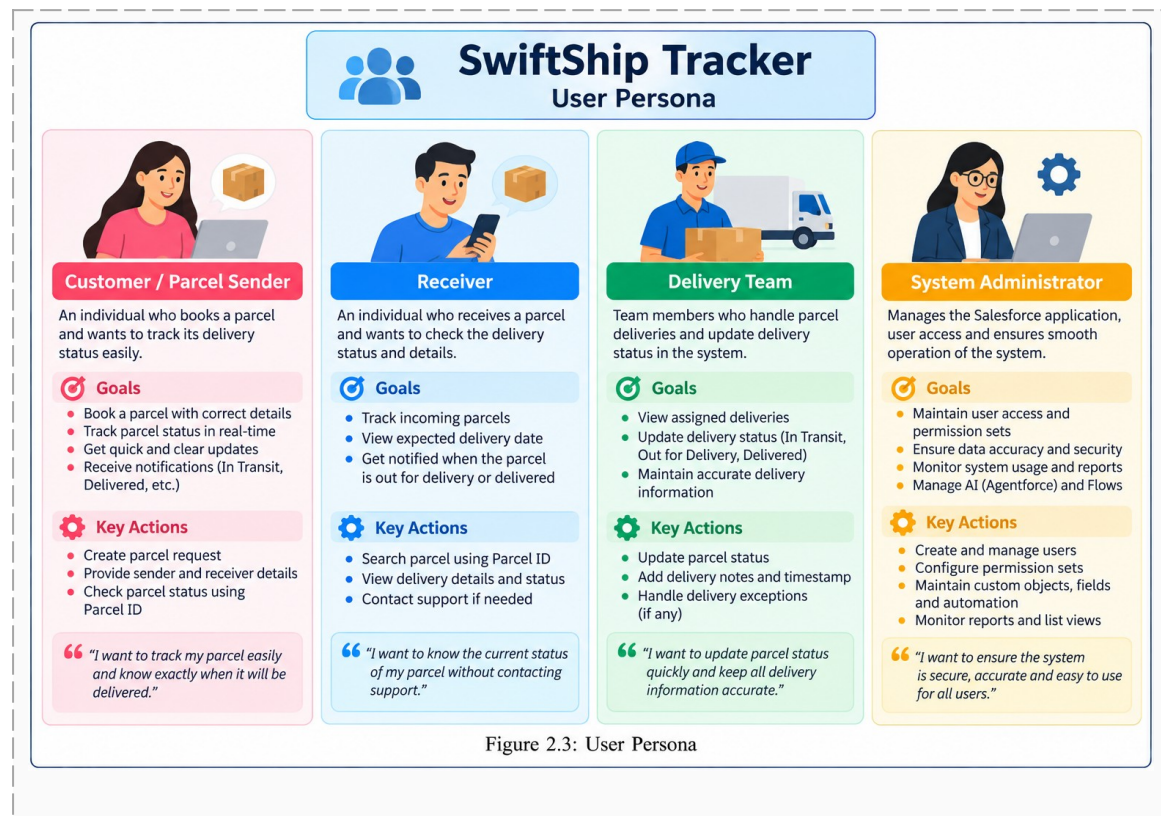
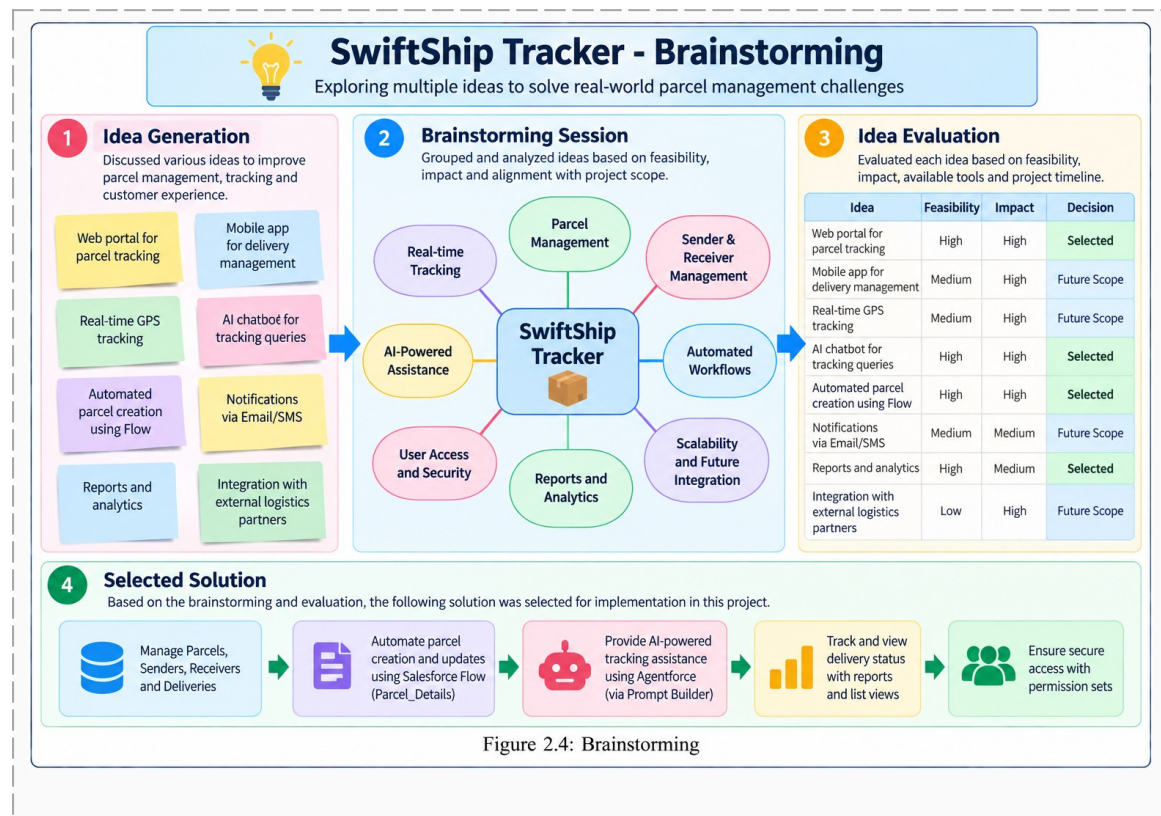


Figure 2.7: User Persona

2.3 Brainstorming and Selection

- Manual record keeping was considered unsuitable for a Salesforce application.
- Spreadsheet tracking was less integrated with Salesforce automation.
- A standalone portal would require additional infrastructure.
- Salesforce was selected because objects, relationships, Flow and AI capabilities can be combined.
- Agentforce and Prompt Builder were selected for conversational retrieval.



3. REQUIREMENT ANALYSIS

Requirements were converted into Salesforce metadata and observable application behavior. The implementation is centered on four custom objects, their fields, relationships, navigation tabs, permissions, Prompt Builder and the Parcel_Details Flow.

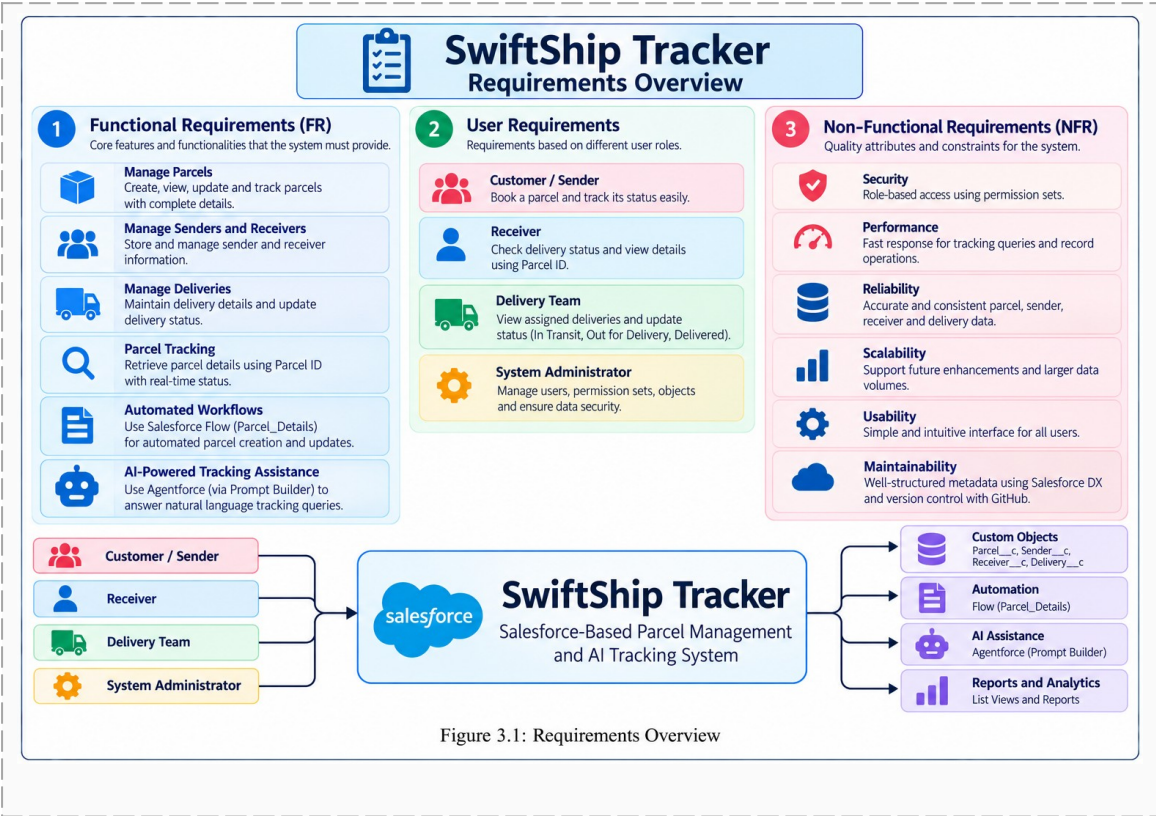


Figure 3.9: Requirements Overview

3.1 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Status
FR-01	Parcel records	Implemented
FR-02	Parcel ID, status, weight, estimated delivery	Implemented
FR-03	Sender records	Implemented
FR-04	Receiver records	Implemented
FR-05	Delivery records	Implemented
FR-06	Custom tabs	Implemented
FR-07	Parcel ID retrieval	Implemented
FR-08	Prompt Builder response	Implemented
FR-09	Flow output	Implemented
FR-10	Permission sets	Implemented
FR-11	GitHub source control	Implemented

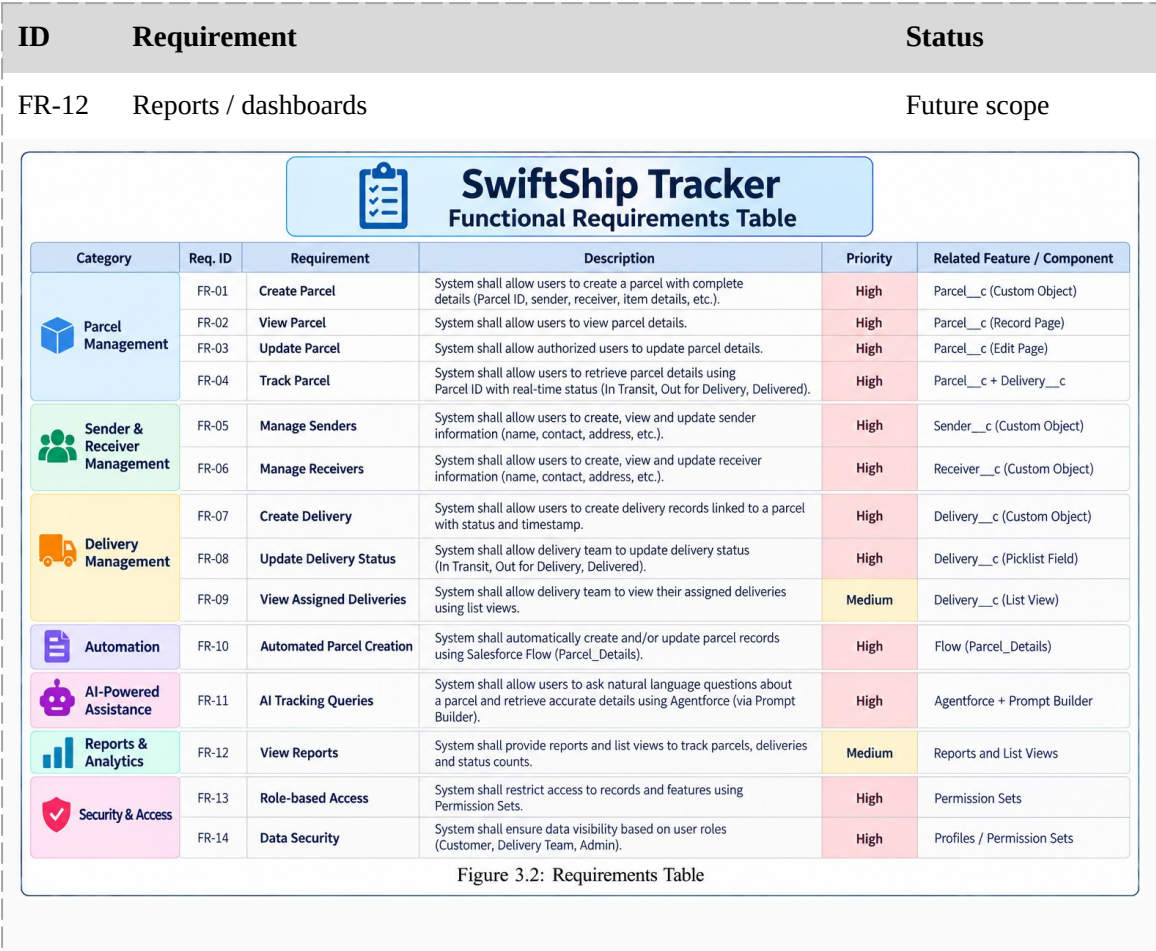


Figure 3.10: Requirements Table

3.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Area	Requirement	Implementation
Usability	Simple parcel lookup	Parcel_ID__c lookup
Security	Controlled access	Permission Sets
Maintainability	Separated metadata	Salesforce DX
Traceability	Source history	GitHub
Reliability	Consistent retrieval	Get Records
AI response	Structured output	Prompt Builder

 SwiftShip Tracker Non-Functional Requirements (NFR) Table					
Category	NFR ID	Non-Functional Requirement	Description	Priority	Evaluation Criteria
 Security	NFR-01	Access Control	The system shall restrict access to data and features based on user roles and permission sets (Customer, Delivery Team, Admin).	High	Verified using Salesforce profiles, permission sets and sharing rules.
	NFR-02	Data Confidentiality	The system shall ensure that sensitive data (e.g., sender and receiver contact information) is protected and visible only to authorized users.	High	Verified using field-level security and record-level access.
 Performance	NFR-03	Response Time	The system shall provide quick response for tracking queries and record operations (create, view, update) within acceptable time limits.	High	Tracking queries should return results within 2-3 seconds.
 Reliability	NFR-04	Data Accuracy	The system shall maintain accurate and consistent data for parcels, senders, receivers and deliveries.	High	Verified through validation rules, required fields and data consistency checks.
 Usability	NFR-05	Ease of Use	The system shall provide a simple and intuitive interface for all user types (Customer, Delivery Team, Admin) to perform their tasks easily.	Medium	Validated through user testing and feedback.
 Scalability	NFR-06	Future Expansion	The system shall be designed to support future enhancements such as additional fields, reports, dashboards and integration with logistics partners.	Medium	Achieved using Salesforce's scalable architecture and customizable components.
 Maintainability	NFR-07	Ease of Maintenance	The system shall follow Salesforce best practices using modular design (custom objects, Flow, Prompt Builder) for easy maintenance and updates.	Medium	Verified through use of standard Salesforce tools and documentation.
 Availability	NFR-08	System Availability	The system shall be available to users with high uptime, leveraging Salesforce's cloud infrastructure.	High	Ensured by Salesforce platform SLA (typically 99.9% uptime).

Figure 3.3: Non-Functional Requirements Table

Figure 3.11: NFR Table

3.3 Data Flow

Implemented flow: User/Agent → Parcel ID → Parcel_Details → Get Parcel Records → Retrieve Parcel Details → Assignment Outputs → Output. The Flow searches Parcel__c where Parcel_ID__c equals the input variable Ids.

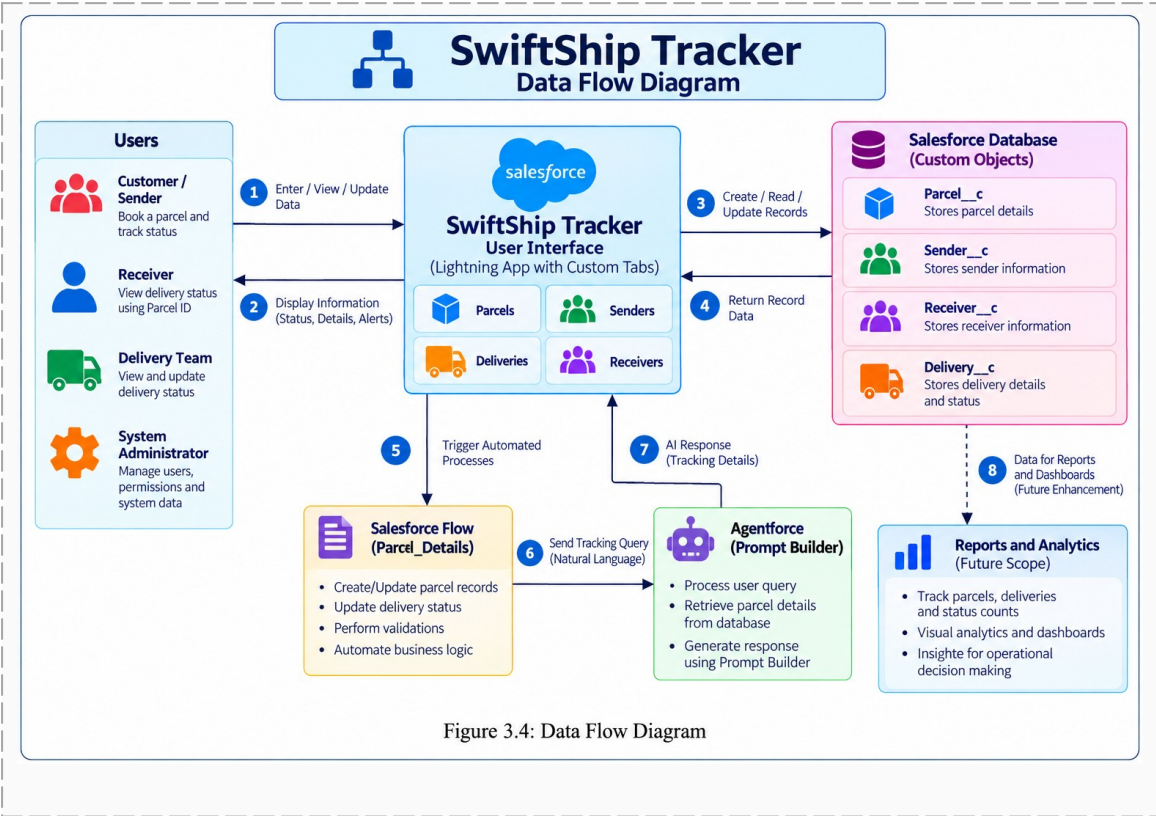


Figure 3.12: Data Flow Diagram

3.4 AI Tracking Flow

The conversational request reaches the configured agent action. The Flow receives the identifier, retrieves the Parcel__c record, passes it to the Prompt Builder action, and returns the generated response.

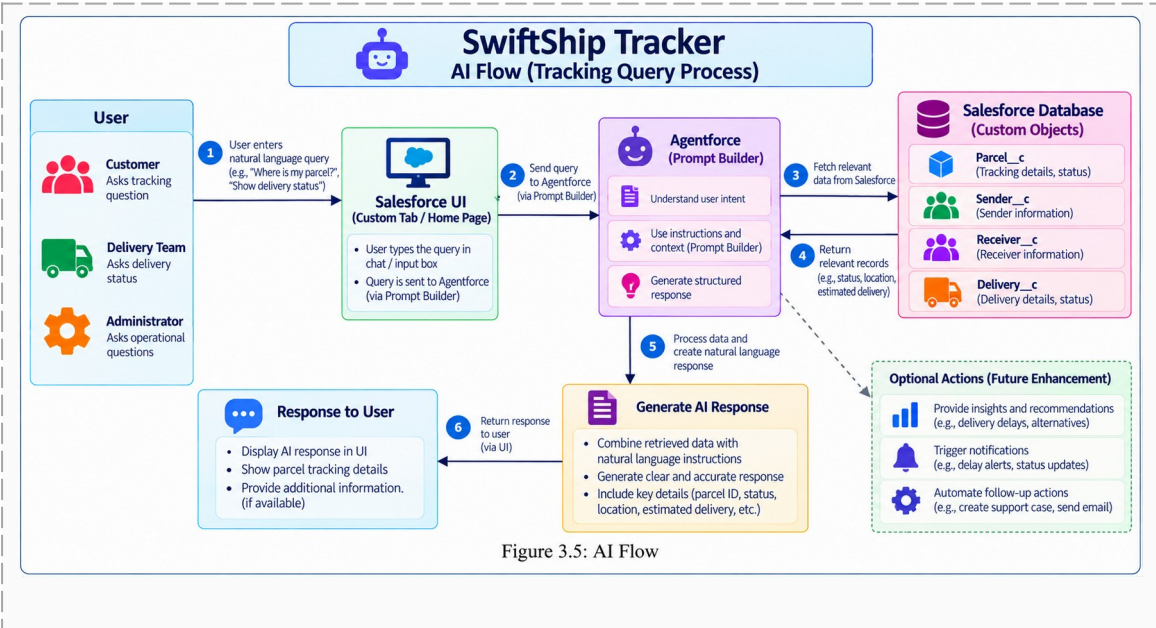
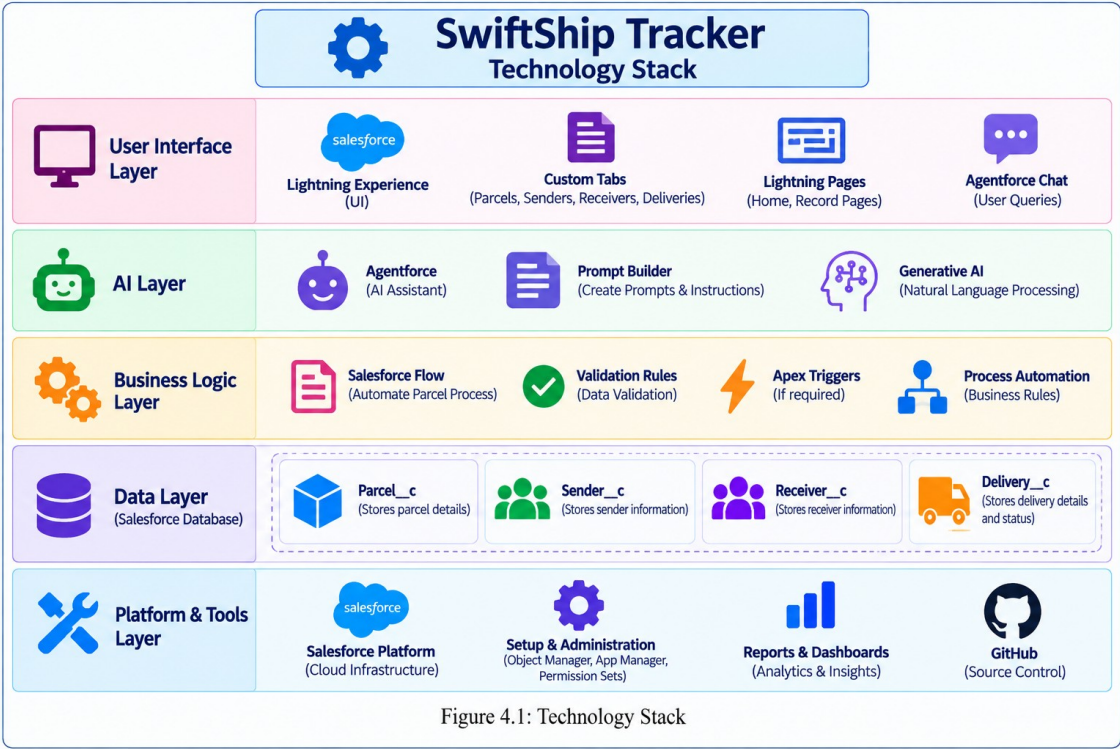


Figure 3.13: AI Flow

4. TECHNOLOGY STACK

Table 4.3: Technology Stack

Layer	Technology	Use
Platform	Salesforce Developer Org	Development
Metadata	Salesforce DX	Source
Database	Custom Objects	Parcel / Delivery / Sender / Receiver
Automation	Flow	Parcel_Details
AI	Prompt Builder / Agentforce	Tracking response
CLI	Salesforce CLI 2.151.7	Validation
Source Control	Git / GitHub	Versioning



The diagram illustrates the technology stack for SwiftShip Tracker, organized into six horizontal layers. At the top is the 'SwiftShip Tracker Technology Stack' header. Below it are the layers: 1. User Interface Layer (purple) containing Lightning Experience (UI), Custom Tabs (Parcels, Senders, Receivers, Deliveries), Lightning Pages (Home, Record Pages), and Agentforce Chat (User Queries). 2. AI Layer (green) containing Agentforce (AI Assistant), Prompt Builder (Create Prompts & Instructions), and Generative AI (Natural Language Processing). 3. Business Logic Layer (orange) containing Salesforce Flow (Automate Parcel Process), Validation Rules (Data Validation), Apex Triggers (If required), and Process Automation (Business Rules). 4. Data Layer (purple) containing the Salesforce Database and four custom objects: Parcel__c (Stores parcel details), Sender__c (Stores sender information), Receiver__c (Stores receiver information), and Delivery__c (Stores delivery details and status). 5. Platform & Tools Layer (blue) containing Salesforce Platform (Cloud Infrastructure), Setup & Administration (Object Manager, App Manager, Permission Sets), Reports & Dashboards (Analytics & Insights), and GitHub (Source Control). The diagram is labeled 'Figure 4.1: Technology Stack' at the bottom.

Figure 4.1: Technology Stack

Figure 4.14: Technology Stack

5. PROJECT DESIGN

The architecture separates data, automation, AI, security and source control.

The data layer is represented by the four custom objects. The automation layer is the Parcel_Details Flow. The AI layer is Retrieve_Parcel_Details. Security is represented by permission sets.

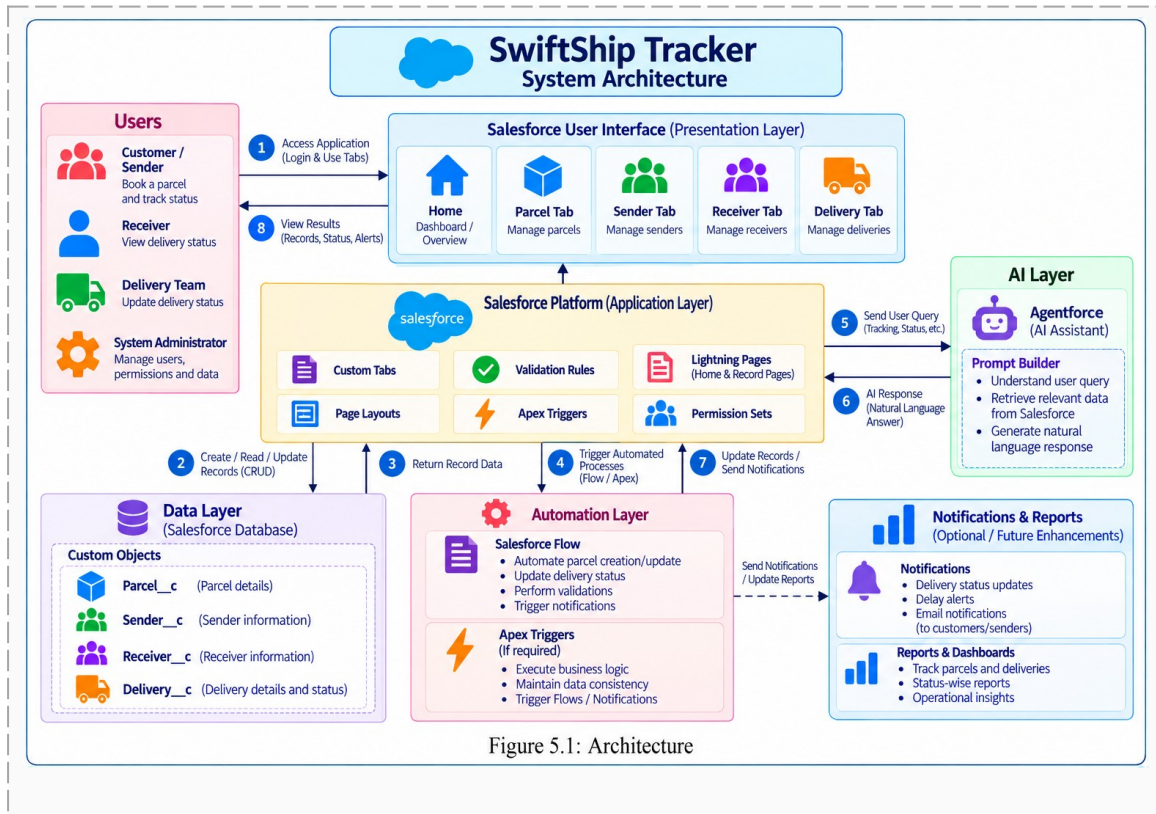


Figure 5.15: Architecture

5.1 Problem–Solution Fit

Table 5.4: Problem–Solution Fit

Problem	Solution	Benefit
Scattered parcel information	Four custom objects	Structured data
Difficult parcel lookup	Parcel_ID__c + Get Records	Direct retrieval
Manual response formatting	Prompt Builder	Consistent response
Access control	Permission Sets	Controlled access
Source traceability	GitHub	Version history

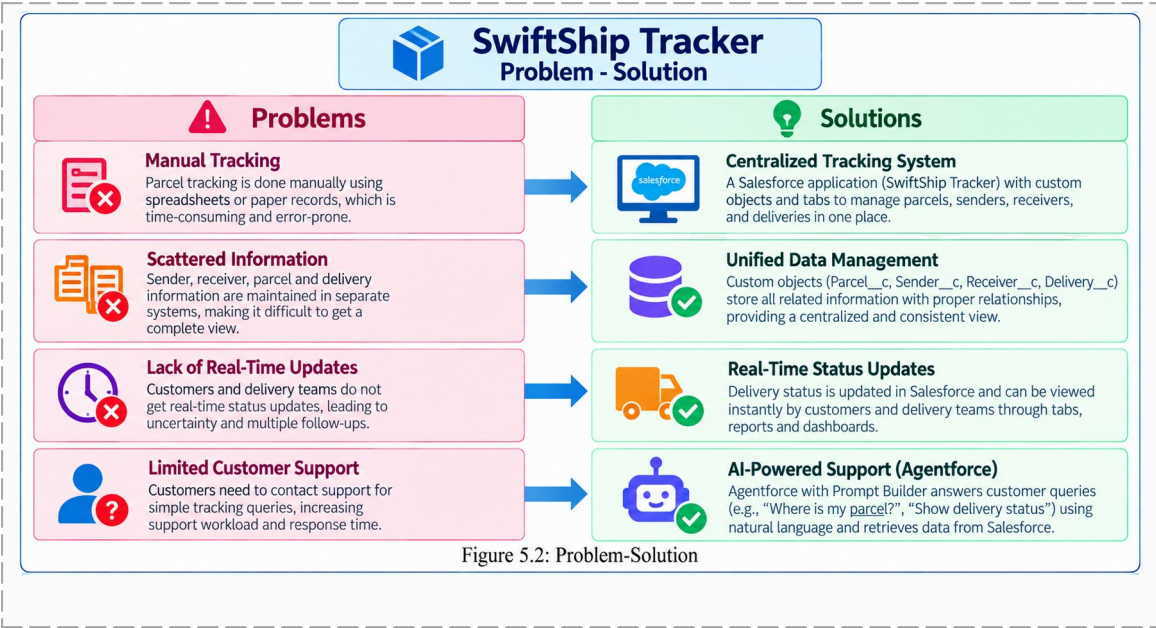


Figure 5.16: Problem-Solution

5.2 Proposed Solution

The solution is a Salesforce DX project deployed to a Salesforce Developer Org. Custom objects provide the data foundation, Flow provides automation, Prompt Builder provides the AI response, and permission sets provide access control.

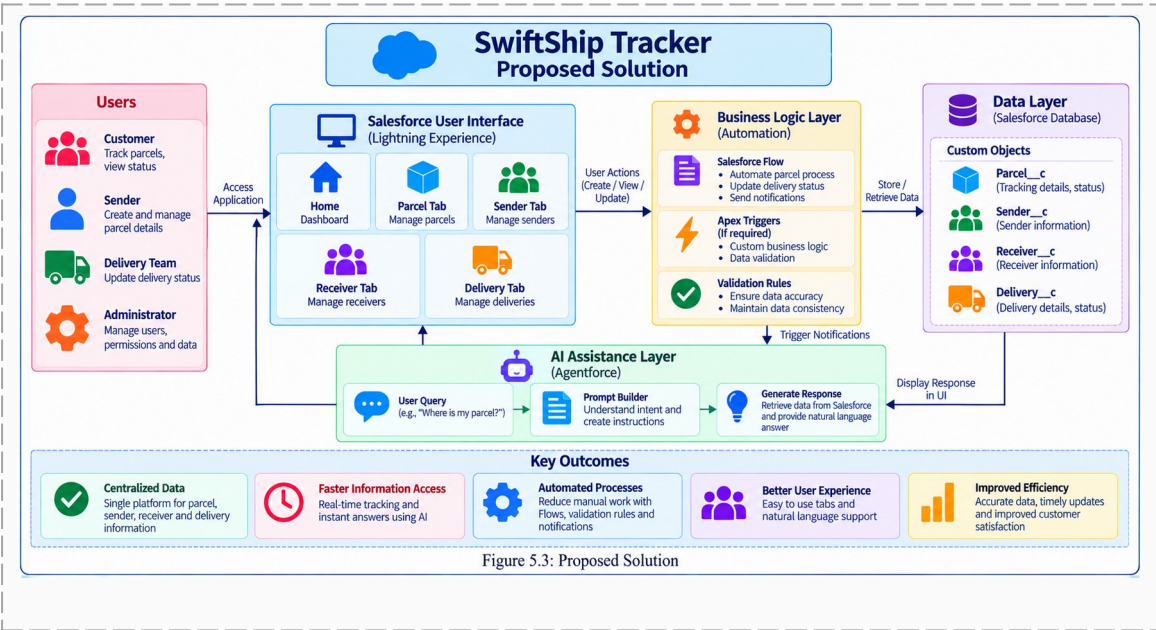


Figure 5.17: Proposed Solution

5.3 Solution Architecture

- User / Agent
- Salesforce user interface
- Parcel, Delivery, Sender and Receiver data layer

- Parcel_Details automation layer
- Prompt Builder / Agentforce AI layer
- Permission-set security layer
- GitHub source-control layer

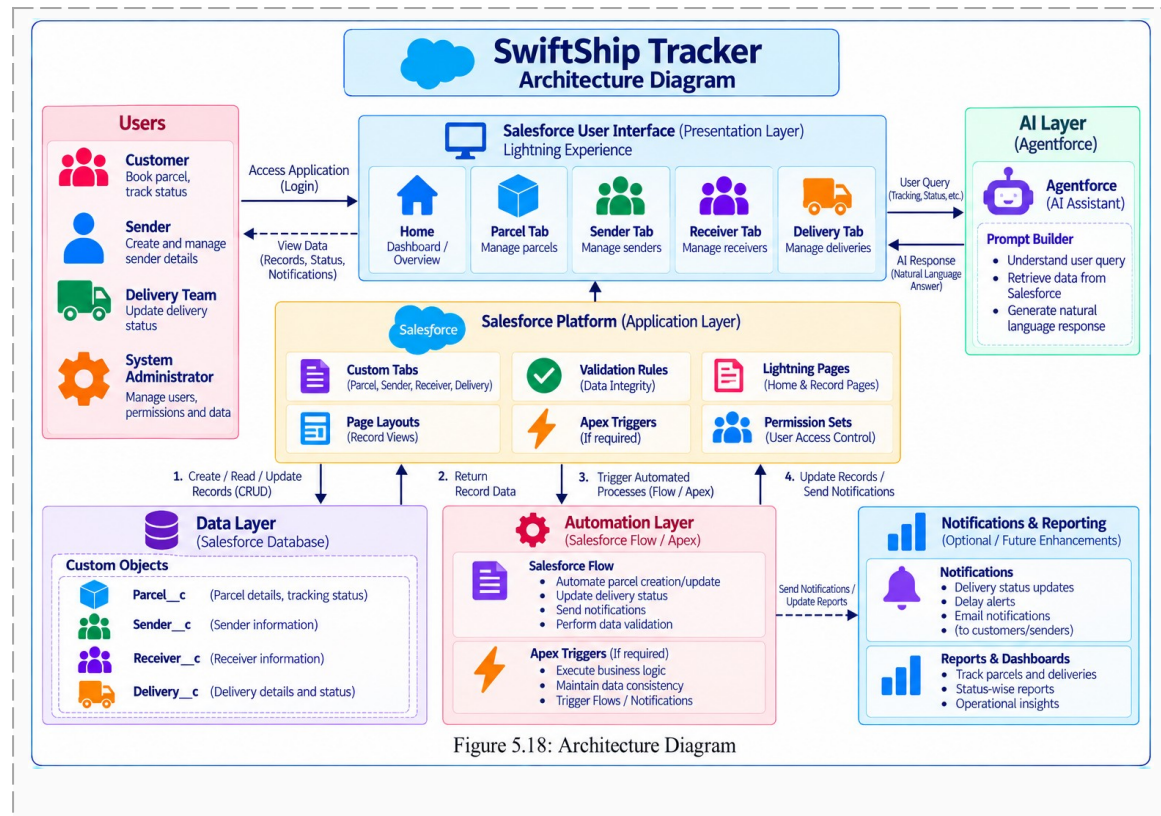


Figure 5.18: Architecture Diagram

6. PROJECT PLANNING

The reference report uses an Agile structure. The actual implementation can be understood as a dependency-based sequence: org authentication, data modeling, fields, tabs, security, Prompt Builder, Flow, validation and GitHub.

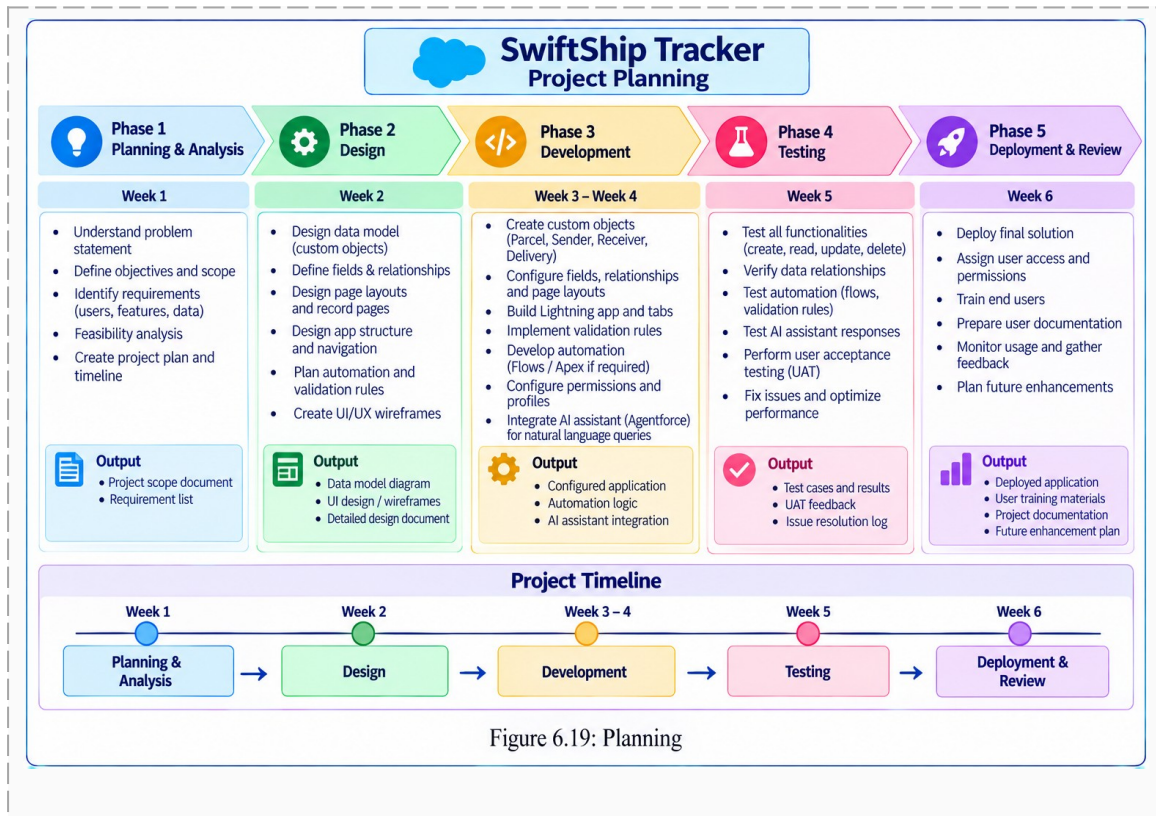


Figure 6.19: Planning

Figure 6.19: Planning

6.1 Work Breakdown

Table 6.5: Work Breakdown

Work Package	Tasks	Output
WP-01	Org authentication	SwiftShipOrg connected
WP-02	Data modeling	Four objects
WP-03	Fields	Custom fields
WP-04	Navigation	Tabs / list views
WP-05	Security	Permission sets
WP-06	AI	Prompt template
WP-07	Automation	Parcel_Details

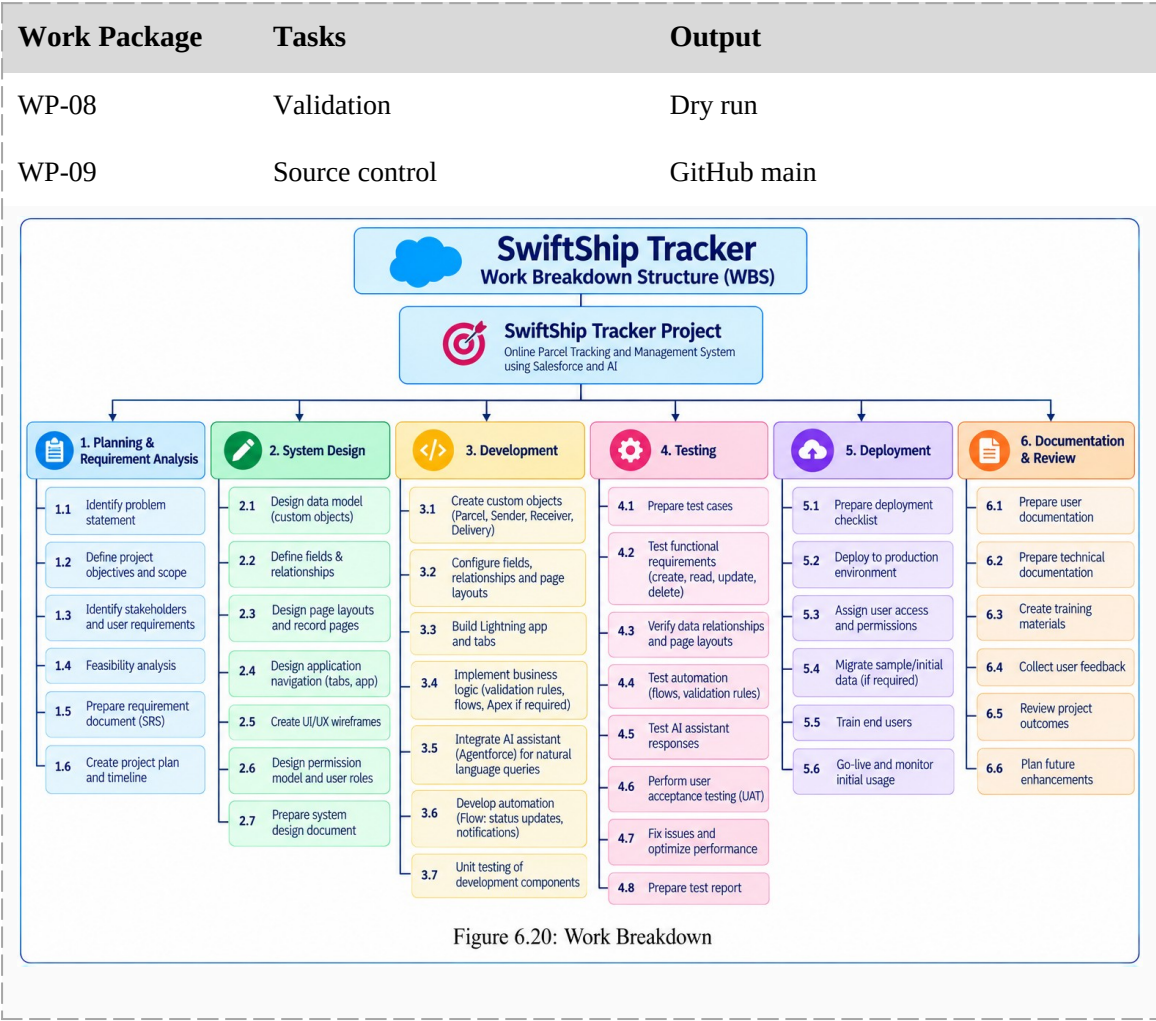


Figure 6.20: Work Breakdown

6.2 Product Backlog, Sprint Schedule and Estimation

Table 6.6: Product Backlog, Sprint Schedule and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Developer Setup	USN-1	As a system administrator, I want to configure and authenticate the Salesforce development environment so that I can build and manage the SwiftShip Tracker application.	3	High	Surya P
Sprint-2	Data Modeling	USN-2	As an administrator, I want to create Parcel, Delivery, Sender, and Receiver objects with appropriate fields and relationships so that shipment information is stored systematically.	5	High	Gopika R
Sprint-2	Data Modeling	USN-3	As a staff member, I want custom tabs and list views for SwiftShip modules so that I can easily navigate and manage parcel and delivery records.	3	High	Kaviya M
Sprint-3	Security and Access	USN-4	As an administrator, I want permission sets and controlled access to SwiftShip data so that users can securely perform their required operations.	3	High	Pavithra A

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-3	Automation	USN-5	As a delivery coordinator, I want the Parcel_Details Flow to retrieve parcel information and process the required outputs so that parcel tracking can be performed with minimal manual effort.	5	High	Swetha M
Sprint-4	AI Tracking	USN-6	As a user, I want Prompt Builder to generate a structured tracking response from retrieved parcel data so that parcel information can be presented clearly.	5	High	Surya P
Sprint-4	AI Tracking	USN-7	As an AI/Agent user, I want Agentforce to invoke the parcel retrieval workflow so that I can obtain parcel tracking information using natural-language queries.	5	High	Gopika R
Sprint-5	Validation and Testing	USN-8	As a developer, I want to validate the Salesforce metadata and Flow configuration so that the implemented SwiftShip Tracker components are verified before deployment.	3	Medium	Kaviya M

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-6	Source Control and Deployment Validation	USN-9	As a developer, I want to maintain the Salesforce DX source code in GitHub and perform CLI validation so that project changes remain traceable and deployable.	3	Medium	Pavithra A

6.3 Sprint Tracker

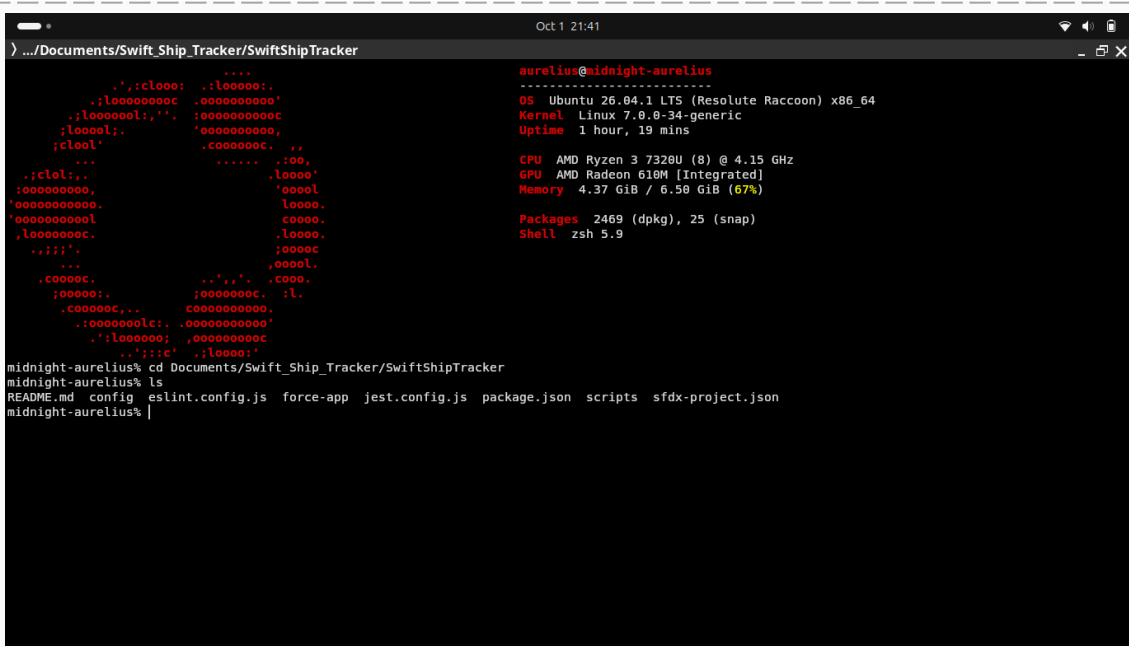
Table 6.7: Sprint Tracker

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	3	1 Day	28/09/2026	28/09/2026	3	28/09/2026
Sprint-2	8	1 Day	29/09/2026	29/09/2026	8	29/09/2026
Sprint-3	8	1 Day	30/09/2026	30/09/2026	8	30/09/2026
Sprint-4	10	1 Day	01/10/2026	01/10/2026	10	01/10/2026
Sprint-5	3	1 Day	02/10/2026	02/10/2026	3	02/10/2026
Sprint-6	3	1 Day	02/10/2026	02/10/2026	3	02/10/2026

7. DEVELOPMENT ENVIRONMENT

The Salesforce DX project is located in the SwiftShipTracker directory.

The project contains README.md, config, eslint configuration, Jest configuration, package.json, scripts, sfdx-project.json and force-app.



```
Oct 1 21:41
~/Documents/Swift_Ship_Tracker/SwiftShipTracker

aurelius@midnight-aurelius
-----
OS Ubuntu 26.04.1 LTS (Resolute Raccoon) x86_64
Kernel Linux 7.0.0-34-generic
Uptime 1 hour, 19 mins

CPU AMD Ryzen 3 7320U (8) @ 4.15 GHz
GPU AMD Radeon 610M [Integrated]
Memory 4.37 GiB / 6.50 GiB (67%)

Packages 2469 (dpkg), 25 (snap)
Shell zsh 5.9

midnight-aurelius% cd Documents/Swift_Ship_Tracker/SwiftShipTracker
midnight-aurelius% ls
README.md config eslint.config.js force-app jest.config.js package.json scripts sfdx-project.json
midnight-aurelius%
```

Figure 7.21: Terminal Project Directory

7.1 Salesforce DX Structure

force-app/main/default contains objects, flows, permissionsets, tabs and other standard Salesforce DX metadata directories. Only metadata actually present in the project is considered implemented.

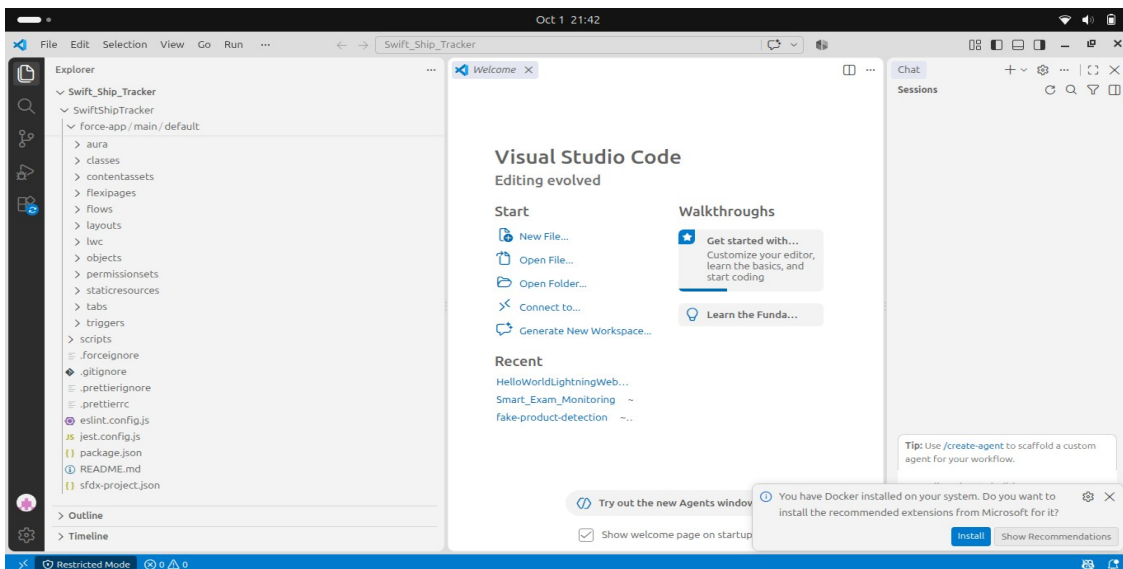


Figure 7.22: VS Code Explorer

7.2 Salesforce Org Authentication

The target org uses the alias SwiftShipOrg.

The successful login command was sf org login web --alias SwiftShipOrg.

Org display confirmed the alias, connected status, API version 67.0, org ID and username.

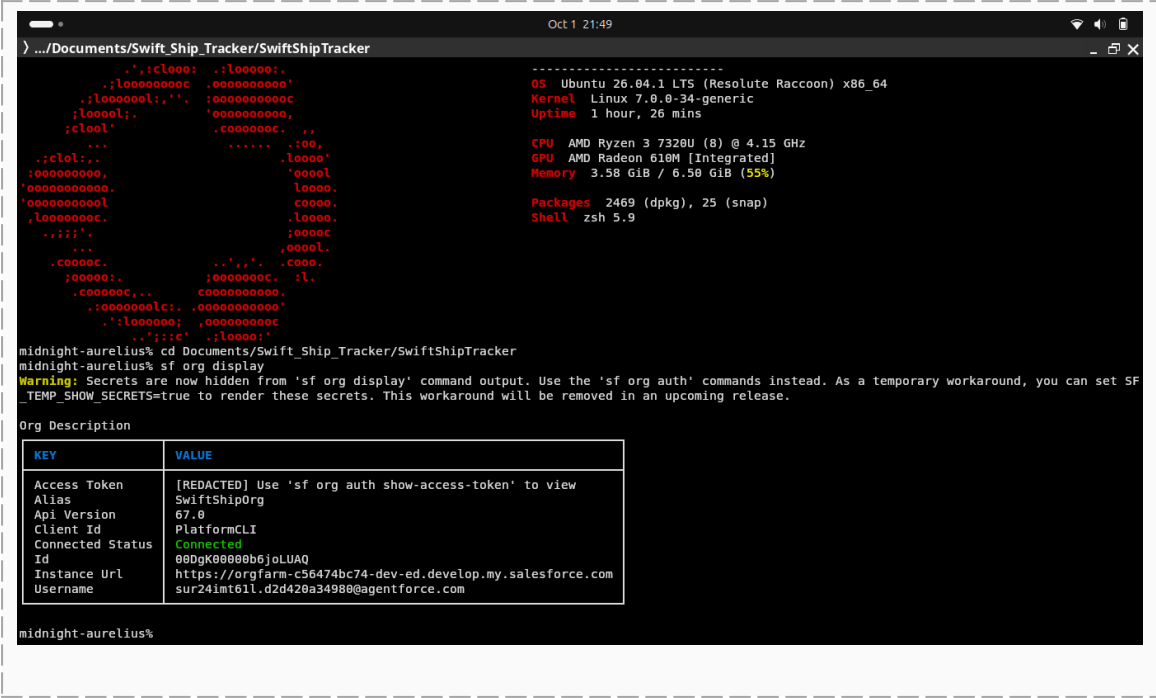
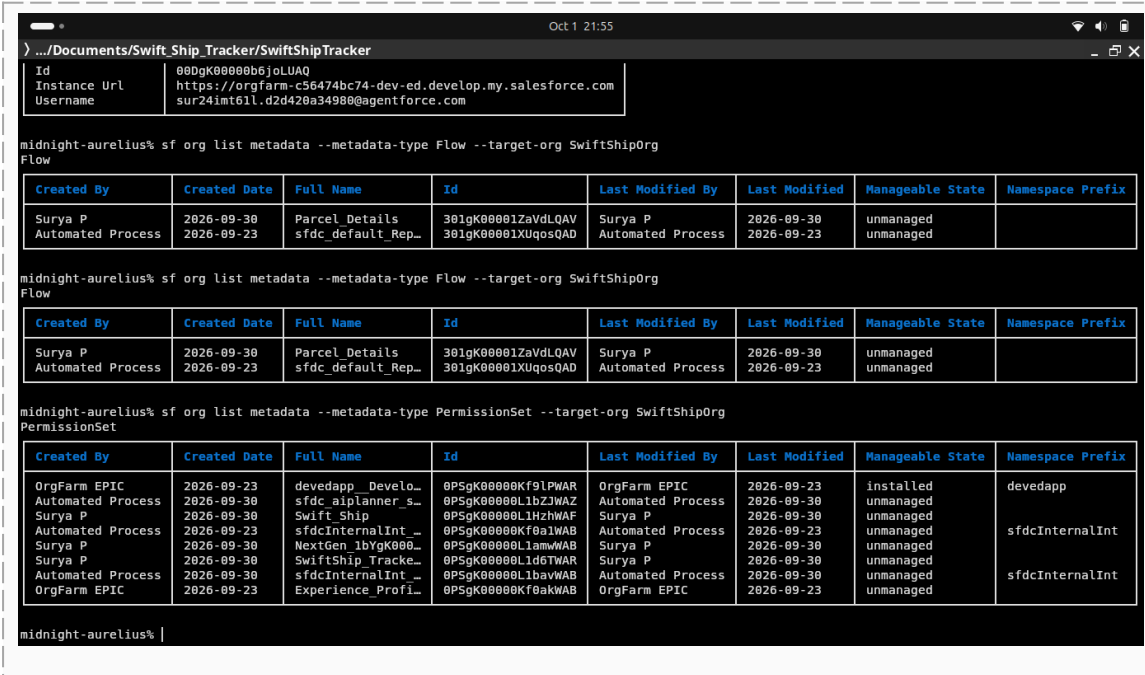


Figure 7.23: SF Org Display

7.3 Org Metadata Verification

Salesforce CLI metadata listing confirmed the Parcel_Details Flow and the project permission sets.

This verification was important because the target org did not support the default source-tracking retrieve workflow.



```

Oct 1 21:55
> .../Documents/Swift_Ship_Tracker/SwiftShipTracker
Id      00DgK00000b6j0LUAQ
Instance Url https://orgfarm-c56474bc74-dev-ed.develop.my.salesforce.com
Username sur24imt61l.d2d420a34980@agentforce.com

midnight-aurelius% sf org list metadata --metadata-type Flow --target-org SwiftShipOrg
Flow


| Created By        | Created Date | Full Name         | Id                 | Last Modified By  | Last Modified | Manageable State | Namespace Prefix |
|-------------------|--------------|-------------------|--------------------|-------------------|---------------|------------------|------------------|
| Surya P           | 2026-09-30   | Parcel_Details    | 301gK00001ZaVdLQAV | Surya P           | 2026-09-30    | unmanaged        |                  |
| Automated Process | 2026-09-23   | sfdc_default_Rep_ | 301gK00001XUqpsQAD | Automated Process | 2026-09-23    | unmanaged        |                  |



midnight-aurelius% sf org list metadata --metadata-type Flow --target-org SwiftShipOrg
Flow


| Created By        | Created Date | Full Name         | Id                 | Last Modified By  | Last Modified | Manageable State | Namespace Prefix |
|-------------------|--------------|-------------------|--------------------|-------------------|---------------|------------------|------------------|
| Surya P           | 2026-09-30   | Parcel_Details    | 301gK00001ZaVdLQAV | Surya P           | 2026-09-30    | unmanaged        |                  |
| Automated Process | 2026-09-23   | sfdc_default_Rep_ | 301gK00001XUqpsQAD | Automated Process | 2026-09-23    | unmanaged        |                  |



midnight-aurelius% sf org list metadata --metadata-type PermissionSet --target-org SwiftShipOrg
PermissionSet


| Created By        | Created Date | Full Name         | Id                 | Last Modified By  | Last Modified | Manageable State | Namespace Prefix |
|-------------------|--------------|-------------------|--------------------|-------------------|---------------|------------------|------------------|
| OrgFarm EPIC      | 2026-09-23   | devedapp_Develo_  | 0PSgK00000Kf9LPWAR | OrgFarm EPIC      | 2026-09-23    | installed        | devedapp         |
| Automated Process | 2026-09-30   | sfdc_aiplanner_s_ | 0PSgK00000L1bZJWAZ | Automated Process | 2026-09-30    | unmanaged        |                  |
| Surya P           | 2026-09-30   | Swift_Ship        | 0PSgK00000L1HzhWAF | Surya P           | 2026-09-30    | unmanaged        |                  |
| Automated Process | 2026-09-23   | sfdcInternalInt _ | 0PSgK00000Kf0aIWAB | Automated Process | 2026-09-23    | unmanaged        | sfdcInternalInt  |
| Surya P           | 2026-09-30   | NextGen_lbygK000_ | 0PSgK00000L1amwWAB | Surya P           | 2026-09-30    | unmanaged        |                  |
| Surya P           | 2026-09-30   | SwiftShip_Tracke_ | 0PSgK00000L1d6TWAR | Surya P           | 2026-09-30    | unmanaged        |                  |
| Automated Process | 2026-09-30   | sfdcInternalInt _ | 0PSgK00000L1bavWAB | Automated Process | 2026-09-30    | unmanaged        | sfdcInternalInt  |
| OrgFarm EPIC      | 2026-09-23   | Experience_Profi_ | 0PSgK00000Kf0akWAB | OrgFarm EPIC      | 2026-09-23    | unmanaged        |                  |



midnight-aurelius%

```

Figure 7.24: Metadata Listing

8. DATA MODEL OVERVIEW

The four custom objects are Parcel__c, Delivery__c, Sender__c and Receiver__c. Parcel__c is the central object used by the AI retrieval workflow.

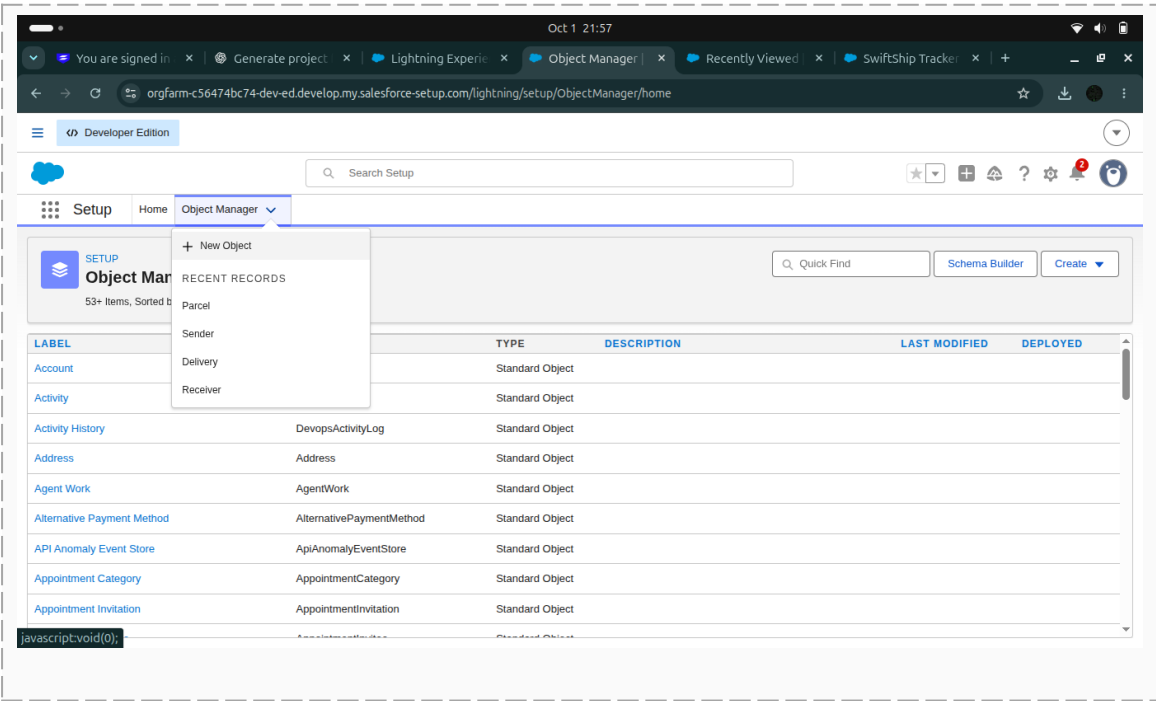


Figure 8.25: Object Manager

8.1 Parcel__c Object

Parcel__c contains Estimated_Delivery_Date__c, Parcel_ID__c, Sender__c, Status__c and Weight__c. Parcel_ID__c is the key field used by the Flow lookup.

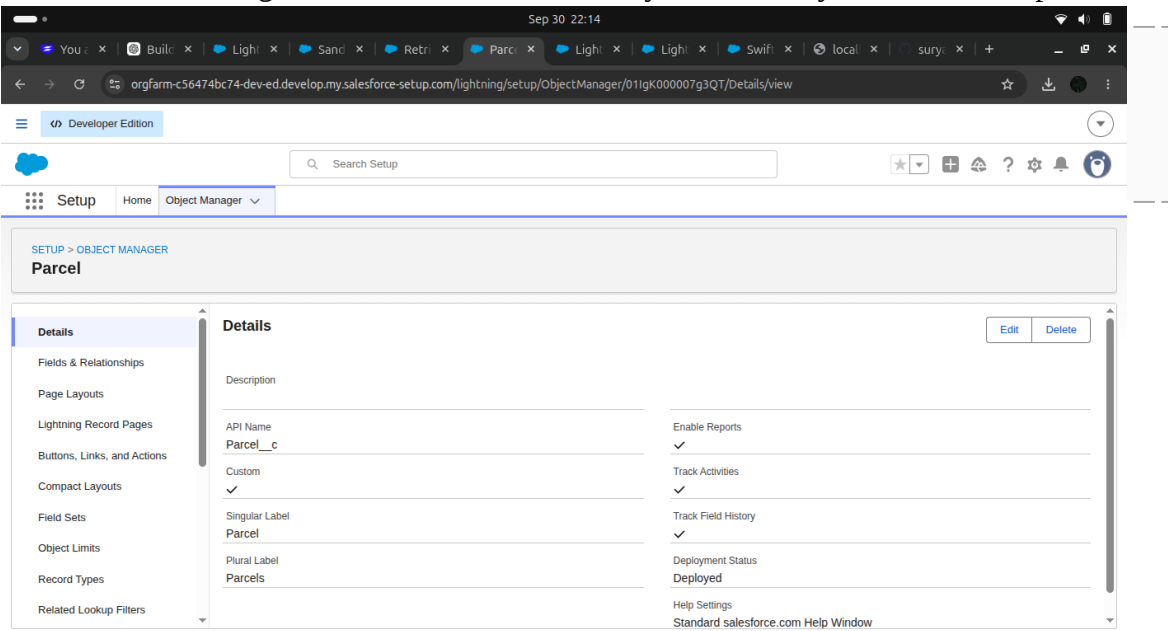


Figure 8.26: Parcel Object

8.2 Parcel Fields

Table 8.8: Parcel Fields

Field	Purpose
Parcel_ID__c	Parcel lookup identifier
Status__c	Parcel status
Weight__c	Parcel weight
Estimated_Delivery_Date__c	Estimated delivery
Sender__c	Sender relationship

Sep 30 22:15

orgfarm-c56474bc74-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/011gk000007g3QT/FieldsAndRelationships/view

Developer Edition

Search Setup

SetupHomeObject Manager

SETUP > OBJECT MANAGER

Parcel

Details

Fields & Relationships

9 Items, Sorted by Field Label

Quick Find

NewDeleted FieldsField DependenciesSet History Tracking

Last Modified By	LastModifiedById	Lookup(User)	
Owner	OwnerId	Lookup(User,Group)	✓
Parcel ID	Parcel_ID__c	Auto Number	
Parcel Name	Name	Text(80)	✓
Sender	Sender__c	Lookup(Sender)	✓
Status	Status__c	Picklist	
Weight	Weight__c	Number(18, 0)	

Figure 8.27: Parcel Fields

8.3 Delivery__c Object

Delivery__c contains Current_Location__c, Estimated_Delivery_Date__c, Parcel__c and Sender__c. Current_Location__c is configured as a geolocation field in the project metadata.

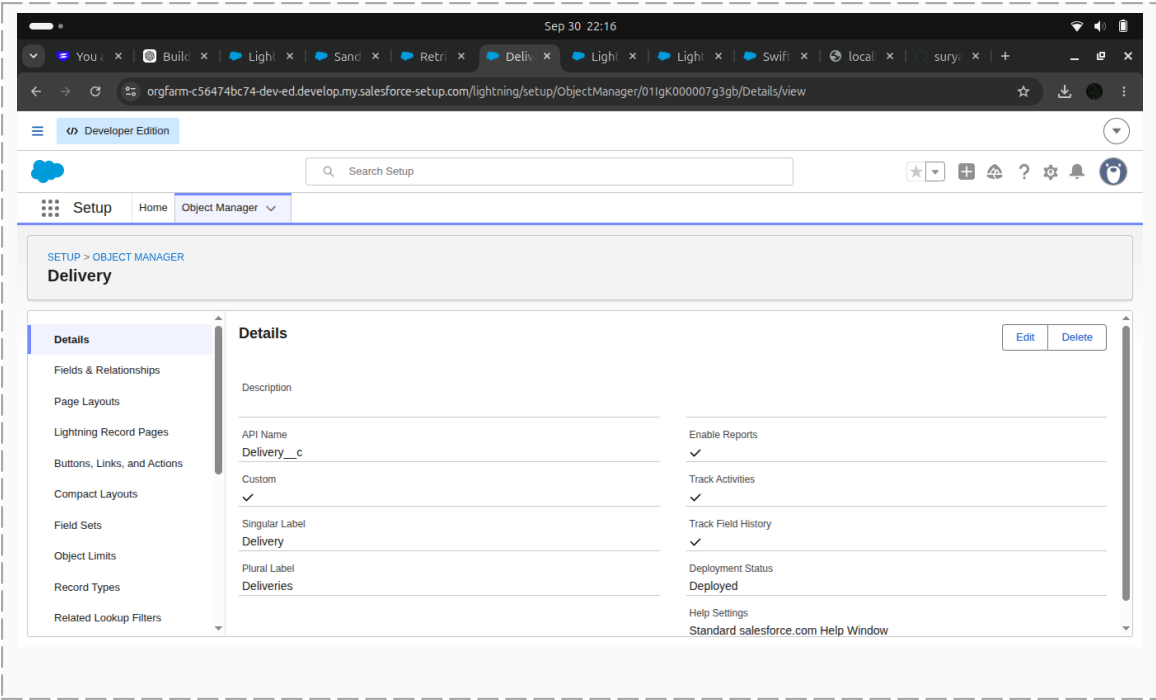


Figure 8.28: Delivery Object

8.4 Delivery Fields

Table 8.9: Delivery Fields

Field	Purpose
Current_Location__c	Current delivery location
Estimated_Delivery_Date__c	Estimated delivery
Parcel__c	Parcel relationship
Sender__c	Sender relationship

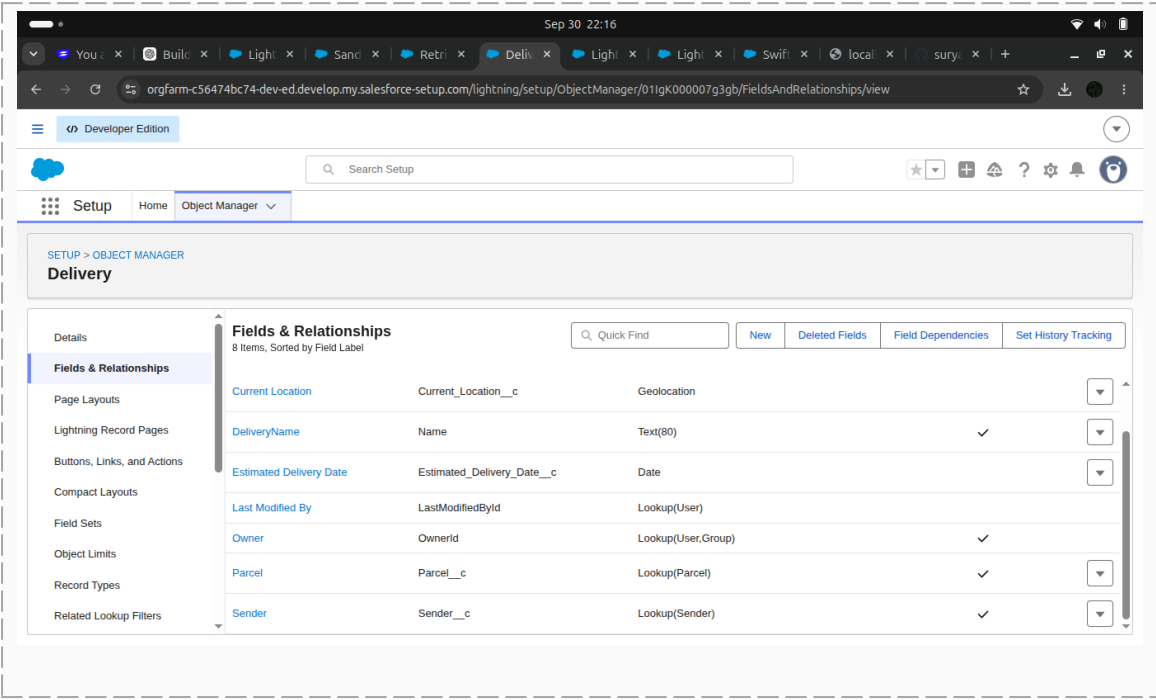


Figure 8.29: Delivery Fields

8.5 Sender__c Object

Sender__c stores Sender_Address__c, Sender_Contact__c and Sender_Email__c. It provides a separate representation of sender information.

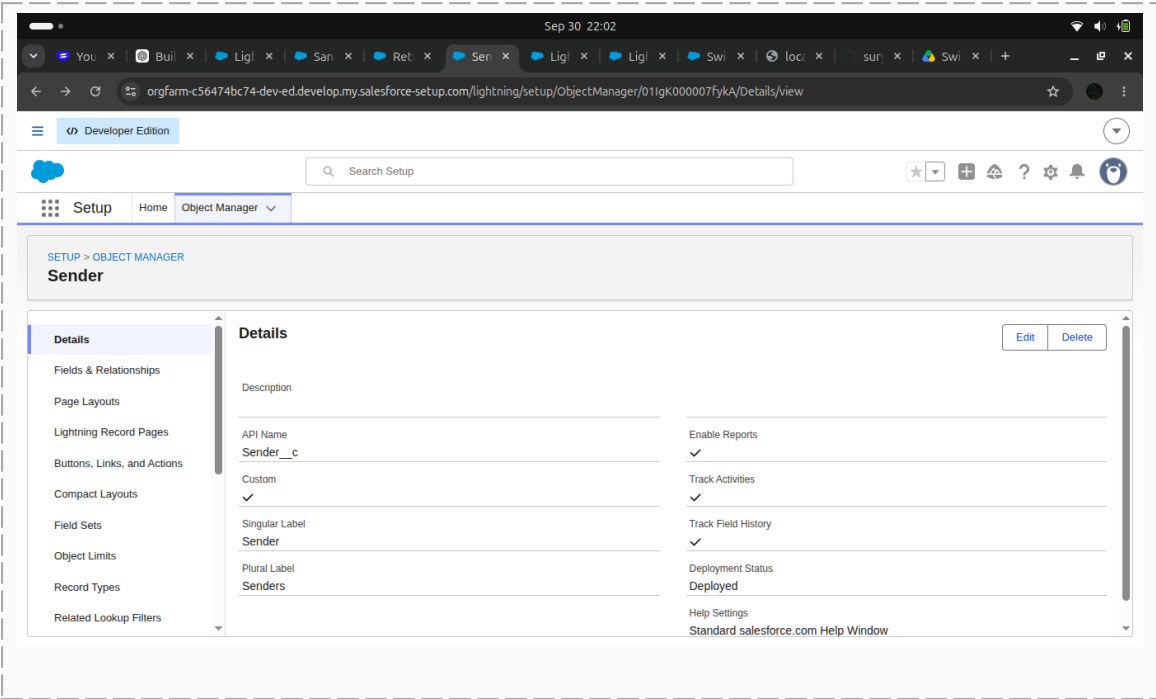


Figure 8.30: Sender Object

8.6 Receiver__c Object

Receiver__c stores Receiver_Address__c, Receiver_Contact__c, Receiver_Email__c, Parcel__c and Sender__c.

It associates receiver information with shipment records.

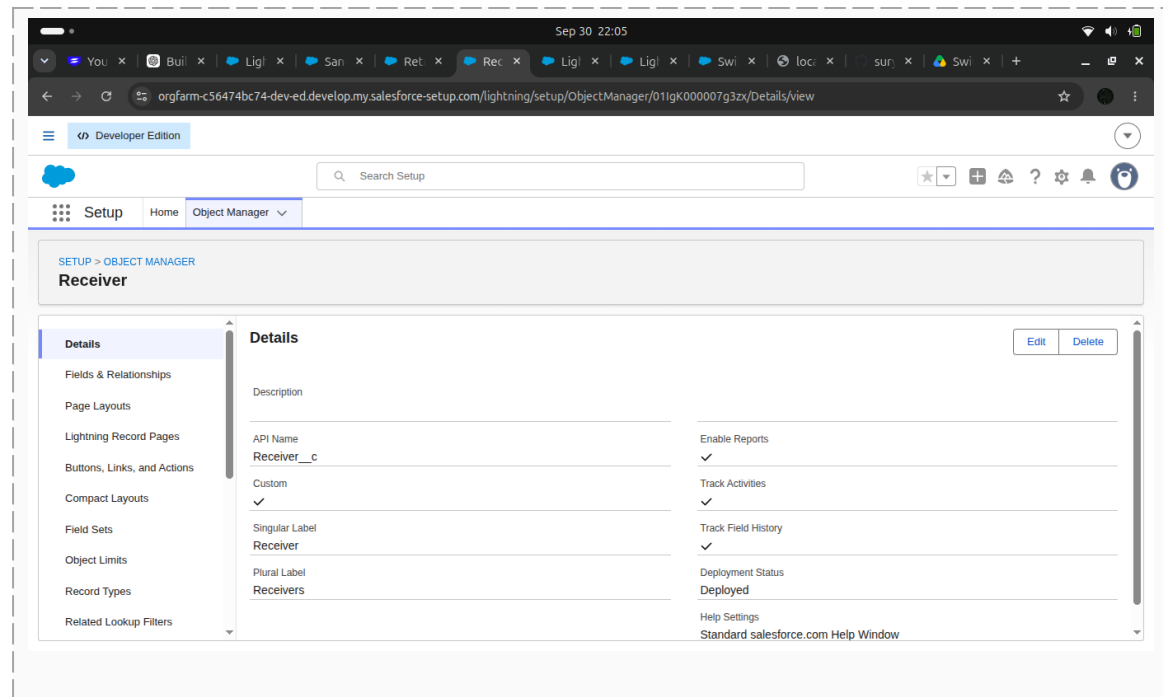


Figure 8.31: Receiver Object

8.7 Object Relationships

Parcel__c references Sender__c. Delivery__c references Parcel__c and Sender__c. Receiver__c references Parcel__c and Sender__c. These relationships keep the model separated across business entities.

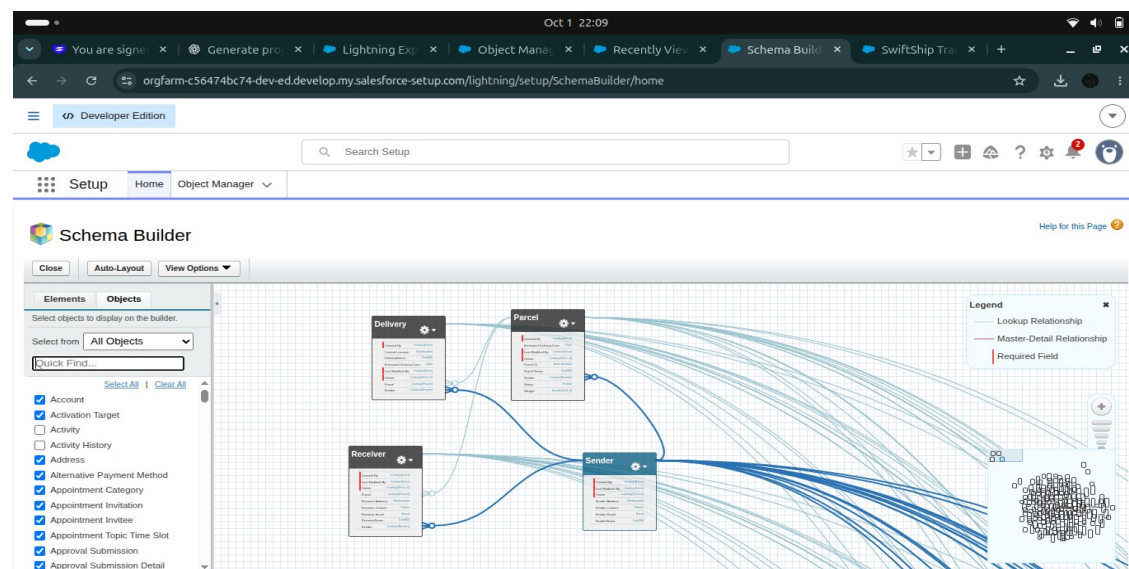


Figure 8.32: Relationship

9. CUSTOM TABS

Four custom tabs exist for Delivery__c, Parcel__c, Receiver__c and Sender__c. These tabs make the custom objects directly accessible from the Salesforce interface.

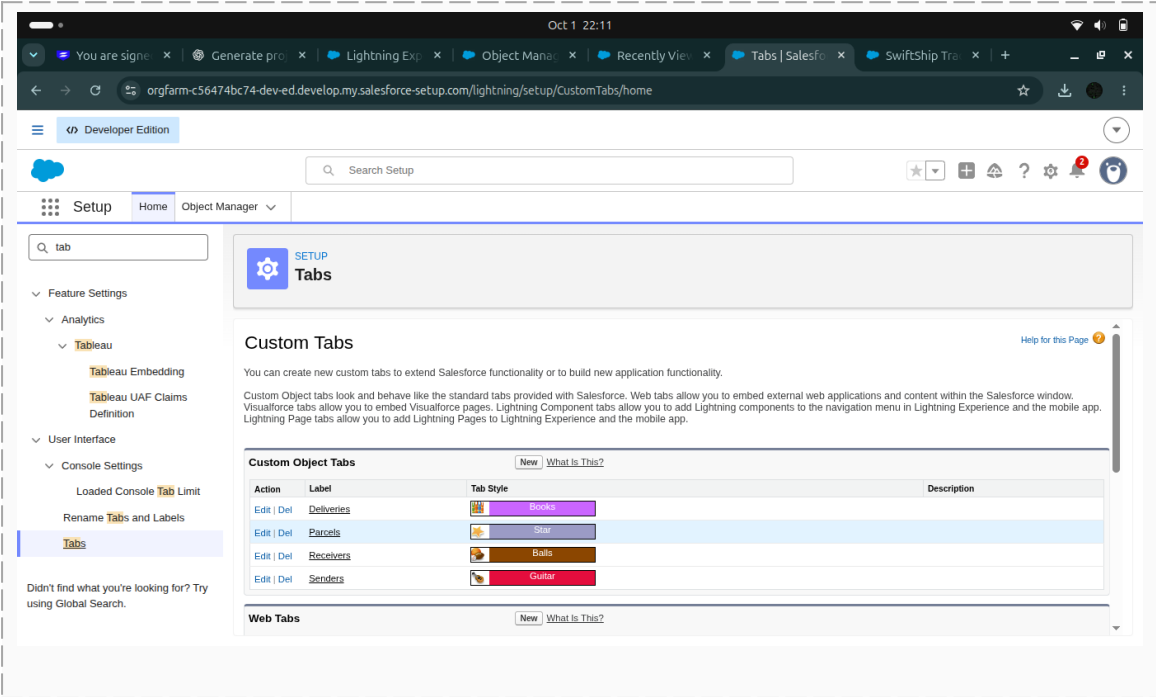


Figure 9.33: Custom Tabs

9.1 List Views

An All list view exists for each of the four custom objects. These views are useful for record inspection and demonstration.

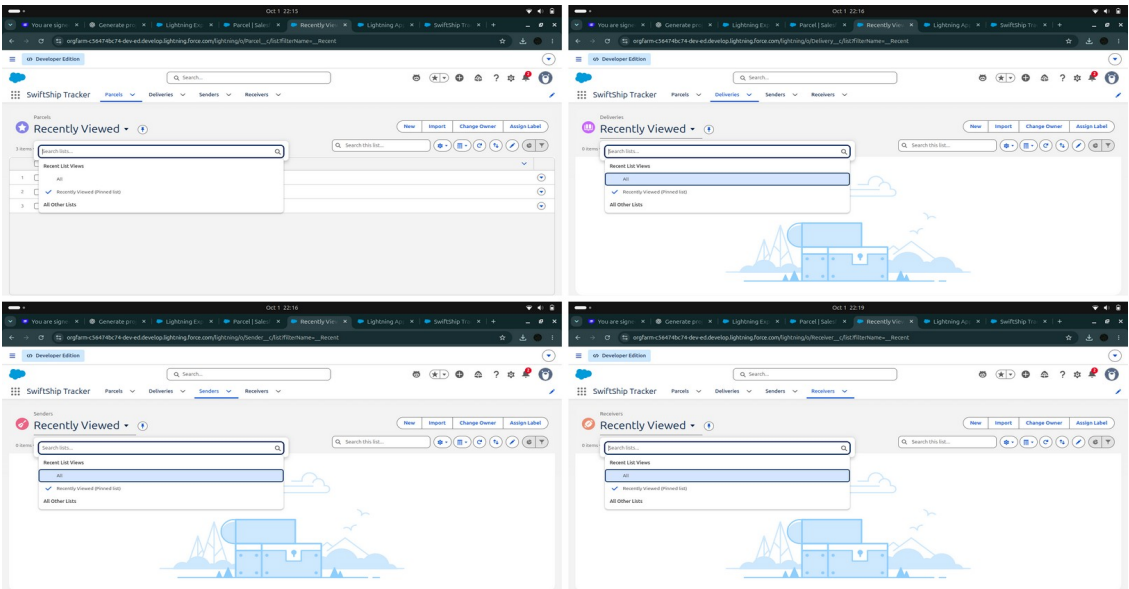


Figure 9.34: List Views

10. SECURITY MODEL

Security is implemented through Salesforce permission sets.

The source contains Swift_Ship and SwiftShip_Tracker1053427577_Permissions.

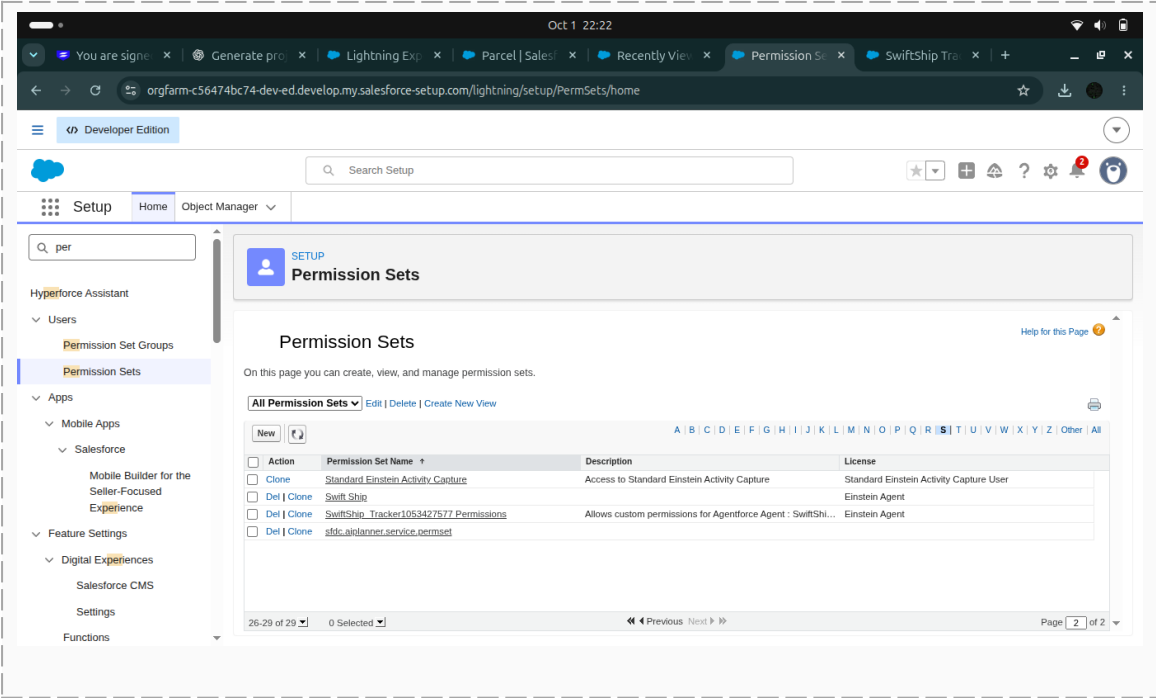


Figure 10.35: Permission Sets

10.1 Swift_Ship Permission Set

Swift_Ship is an unmanaged permission set created by the project user and included in the Salesforce DX source.

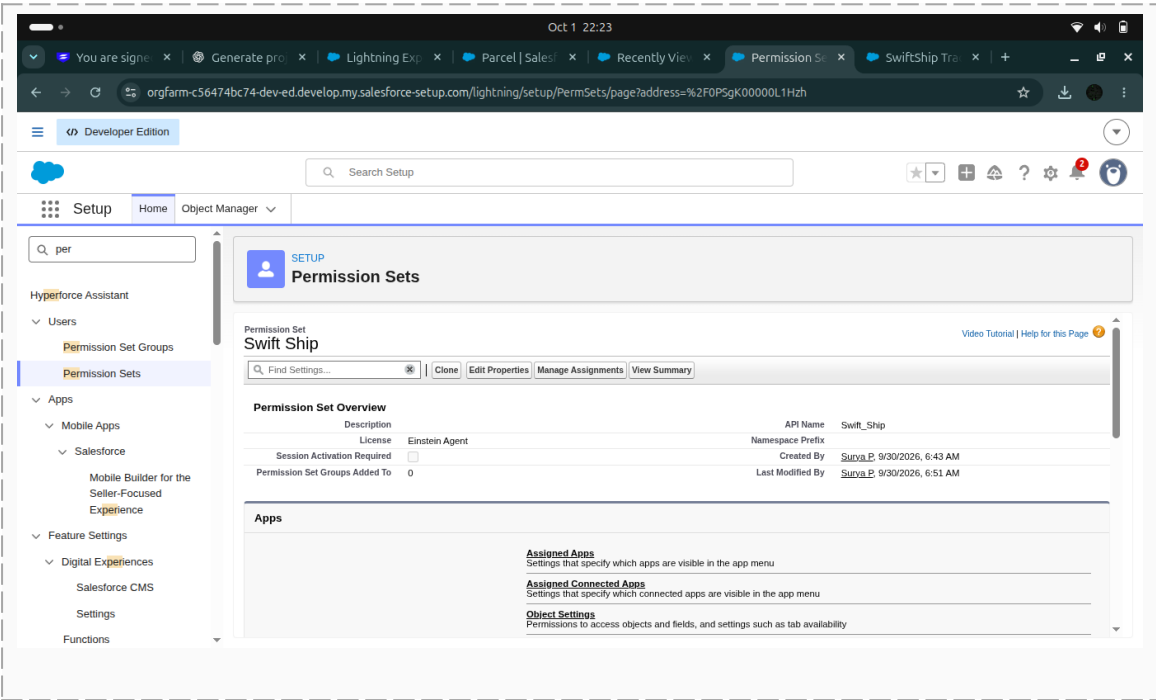


Figure 10.36: Swift_Ship

10.2 SwiftShip Tracker Permissions

SwiftShip_Tracker1053427577_Permissions is another unmanaged project permission set and is included in the deployment source.

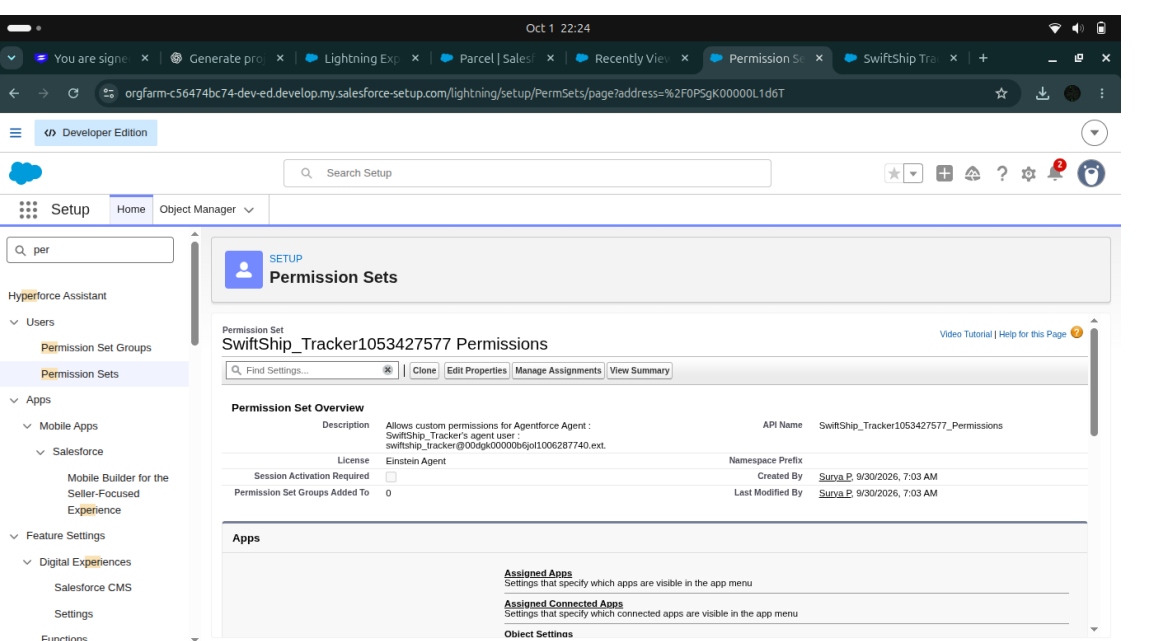


Figure 10.37: SwiftShip Tracker Permission

11. PROMPT BUILDER

The Prompt Builder implementation is named Retrieve_Parcel_Details.

The Flow invokes this action through Parcel_Updates.

The action receives the retrieved Parcel__c record.

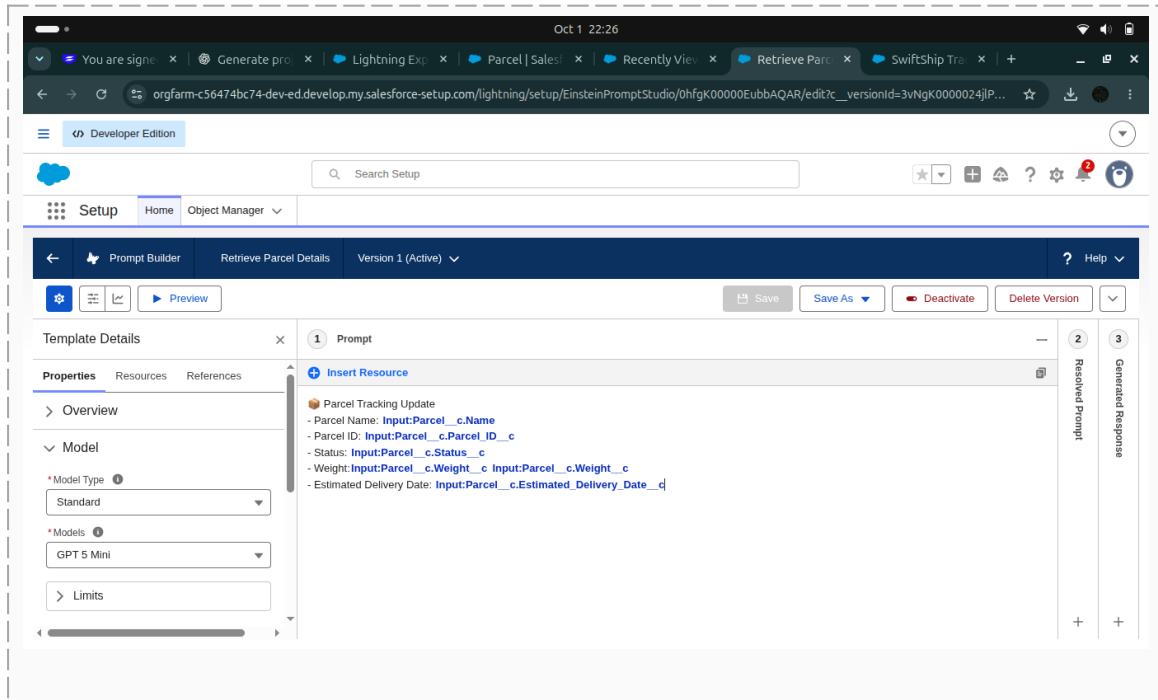


Figure 11.38: Prompt Builder

11.1 Prompt Resources

The prompt is designed around Parcel__c values including Name, Parcel_ID__c, Status__c, Weight__c and Estimated_Delivery_Date__c. The resource mapping ensures that the response is grounded in Salesforce record data.

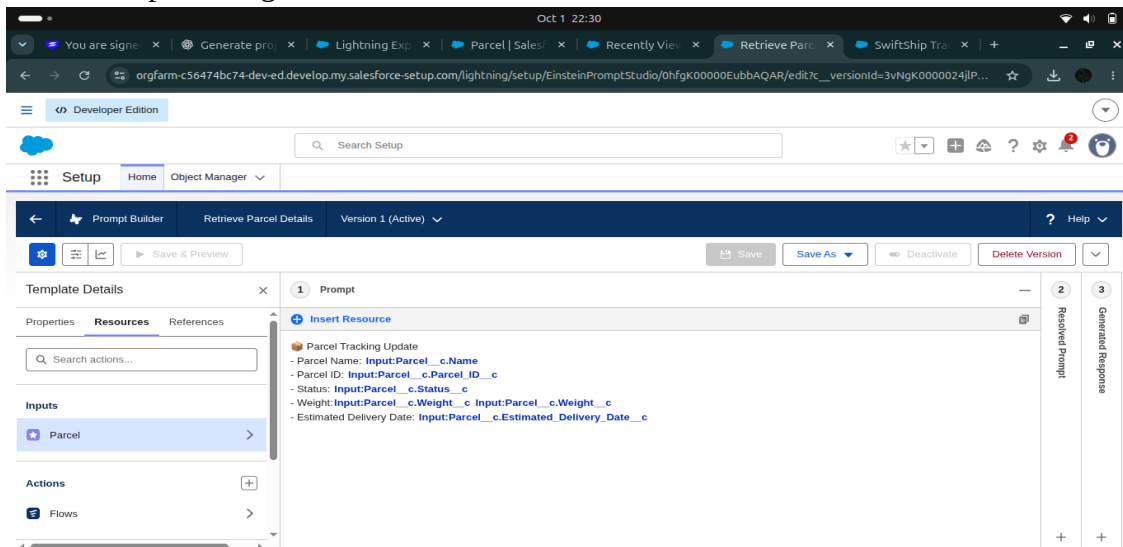


Figure 11.39: Prompt Resource Mapping

11.2 Prompt Response Format

The intended tracking response contains Parcel Name, Parcel ID, Status, Weight and Estimated Delivery Date.

A predictable format improves readability.

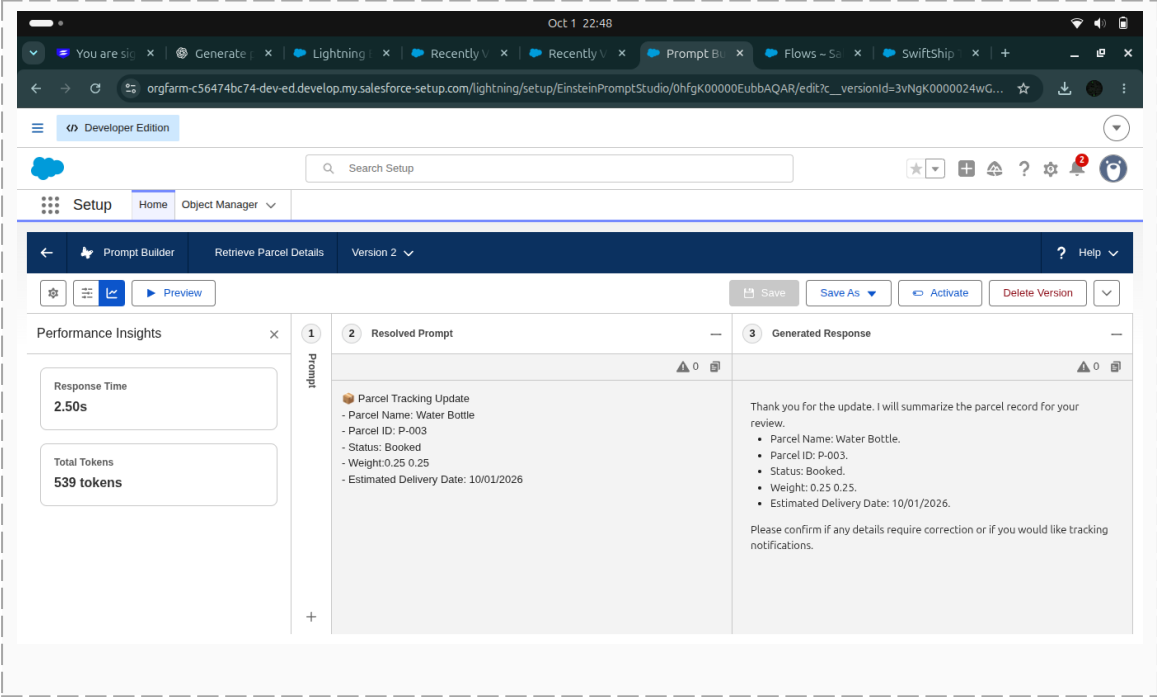


Figure 11.40: Prompt Preview

12. PARCEL_DETAILS FLOW

Parcel_Details is an Auto-Launched Flow with API version 67.0 and Active status.

It performs record retrieval, invokes the prompt action and assigns the generated response.

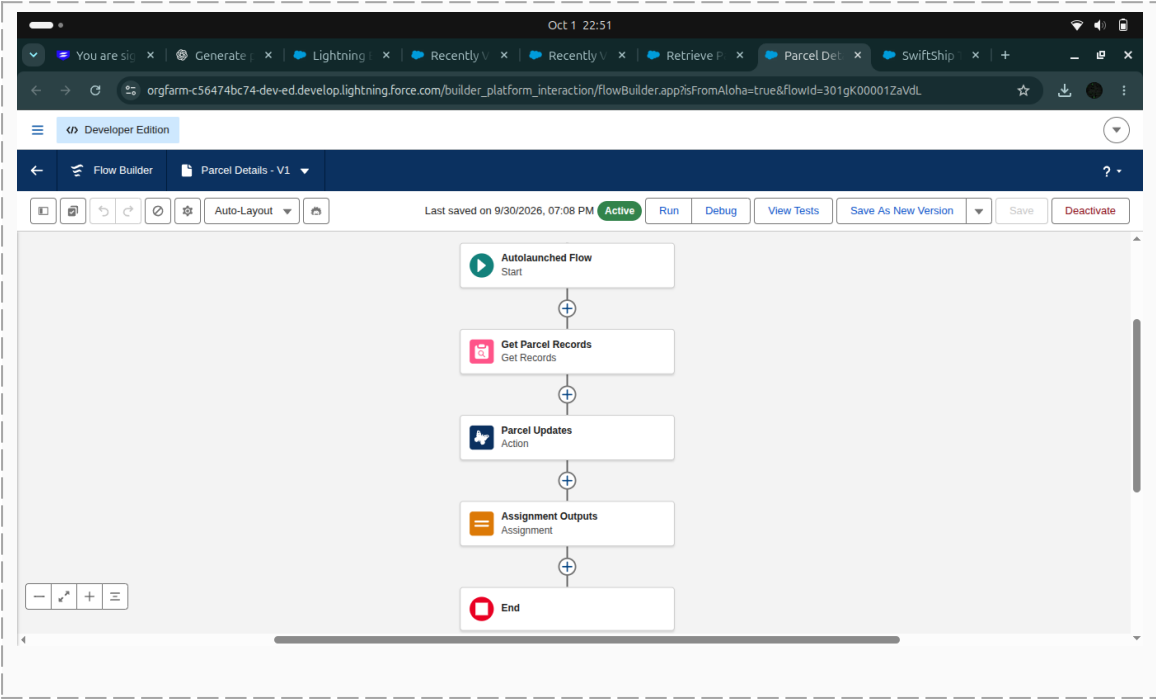


Figure 12.41: Flow Canvas

12.1 Flow Variables

Table 12.10: Flow Variables

Variable	Type	Direction	Purpose
Ids	String	Input	Parcel identifier
Output	String	Output	Generated response

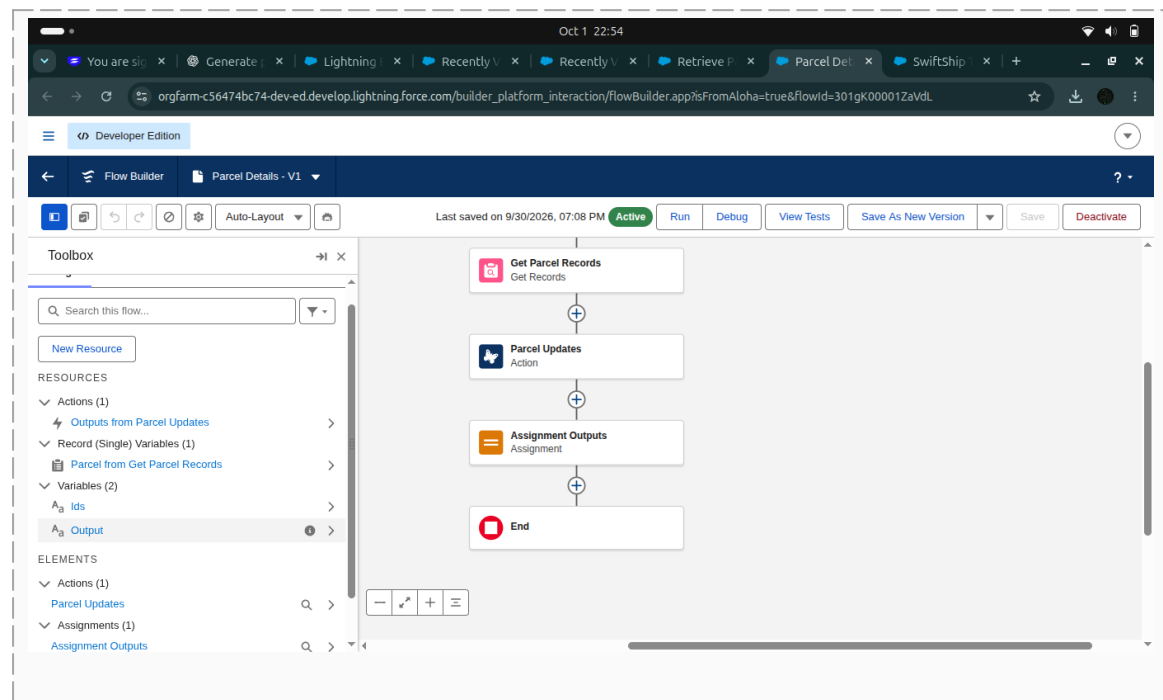


Figure 12.42: Flow Variables

12.2 Get Parcel Records

Get_Parcel_Records is a Record Lookup on Parcel__c.

The filter is Parcel_ID__c Equals Ids and the Flow is configured to retrieve the first matching record.

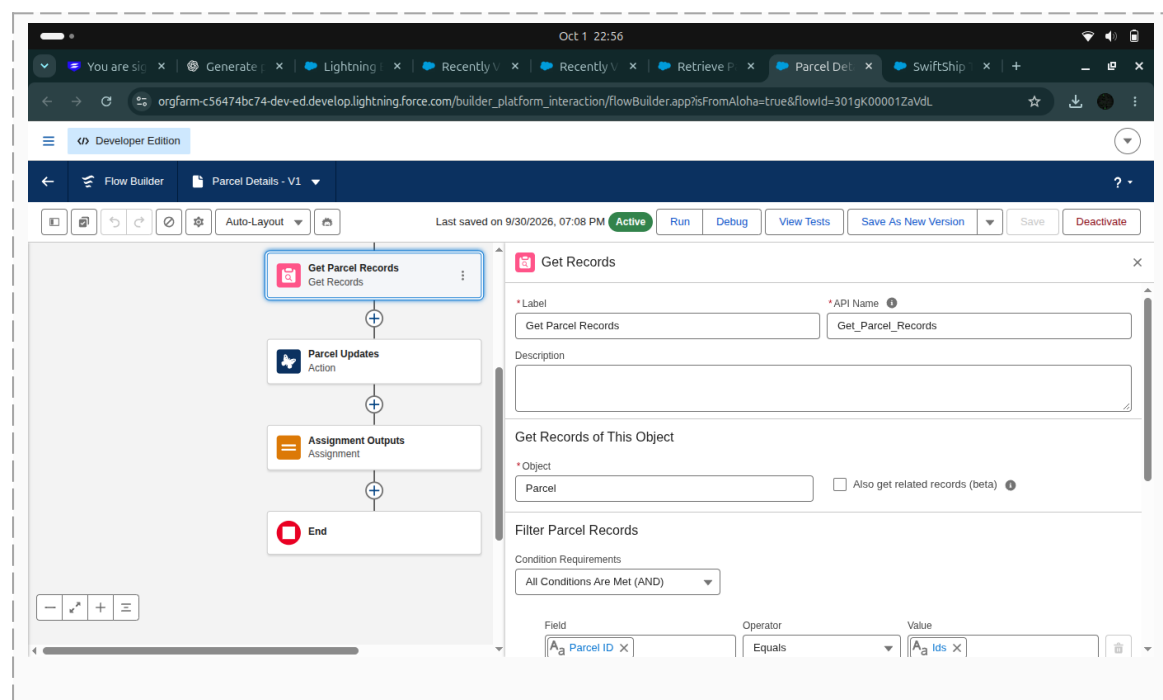


Figure 12.43: Get Records

12.3 Parcel Updates Action

Parcel_Updates is a generatePromptResponse action using Retrieve_Parcel_Details.

The Get_Parcel_Records result is supplied to the action as its input.

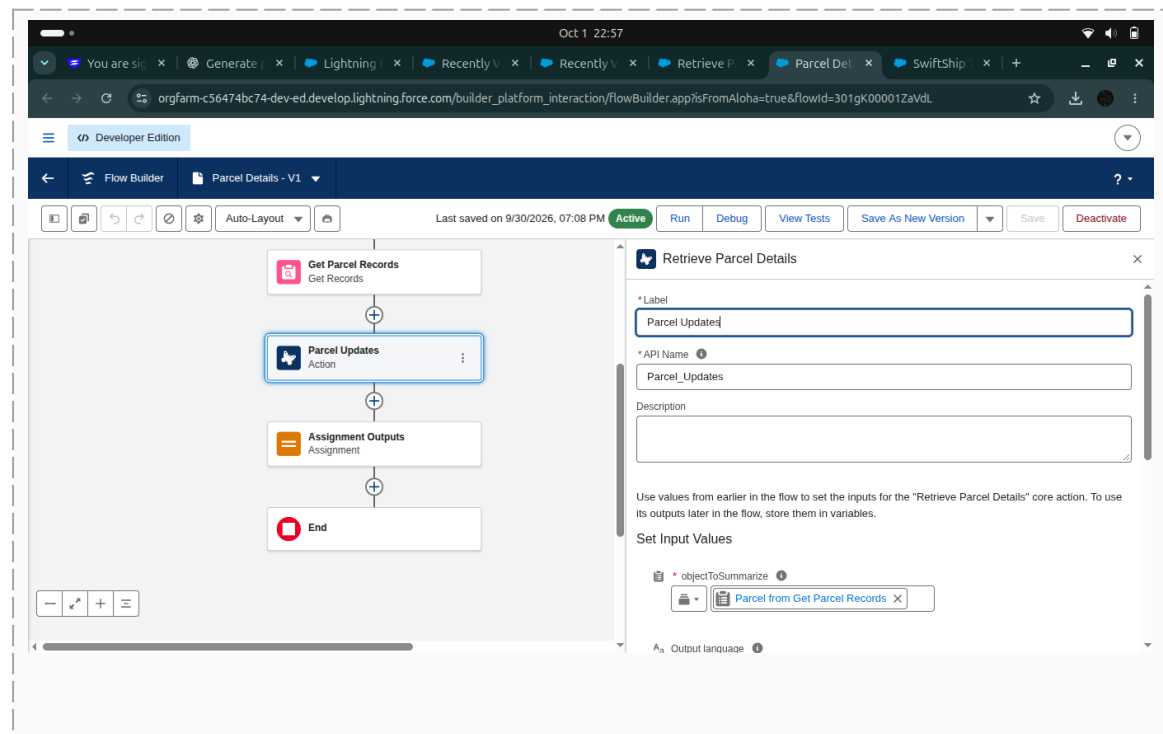


Figure 12.44: Flow Action

12.4 Assignment Outputs

Assignment_Outputs assigns Parcel_Updates.promptResponse to Output. This exposes the AI-generated result as the Flow output.

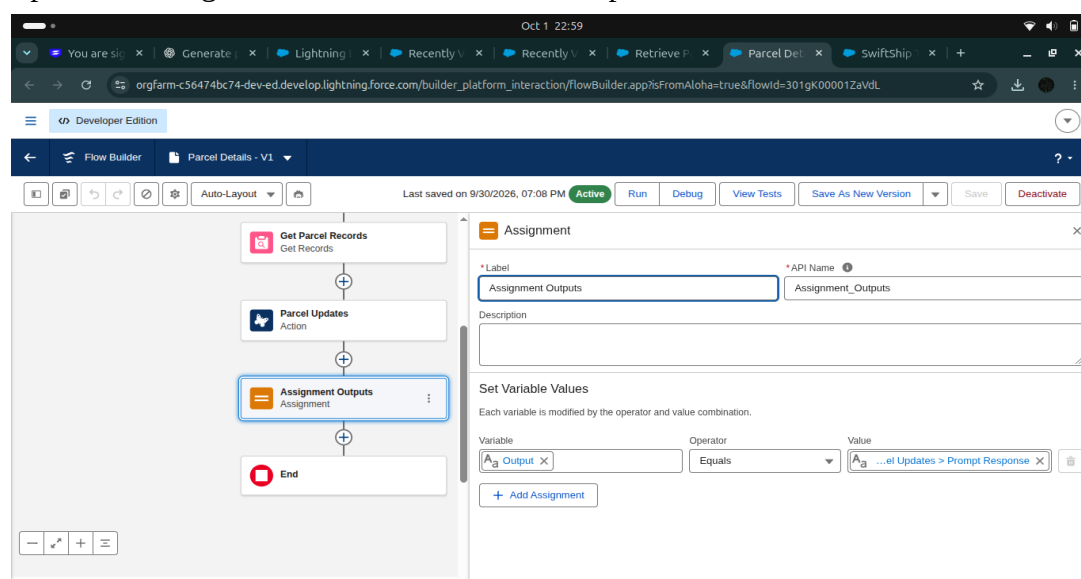


Figure 12.45: Assignment

12.5 Flow Sequence

The exact sequence is Start → Get Parcel Records → Parcel Updates → Assignment Outputs → End.

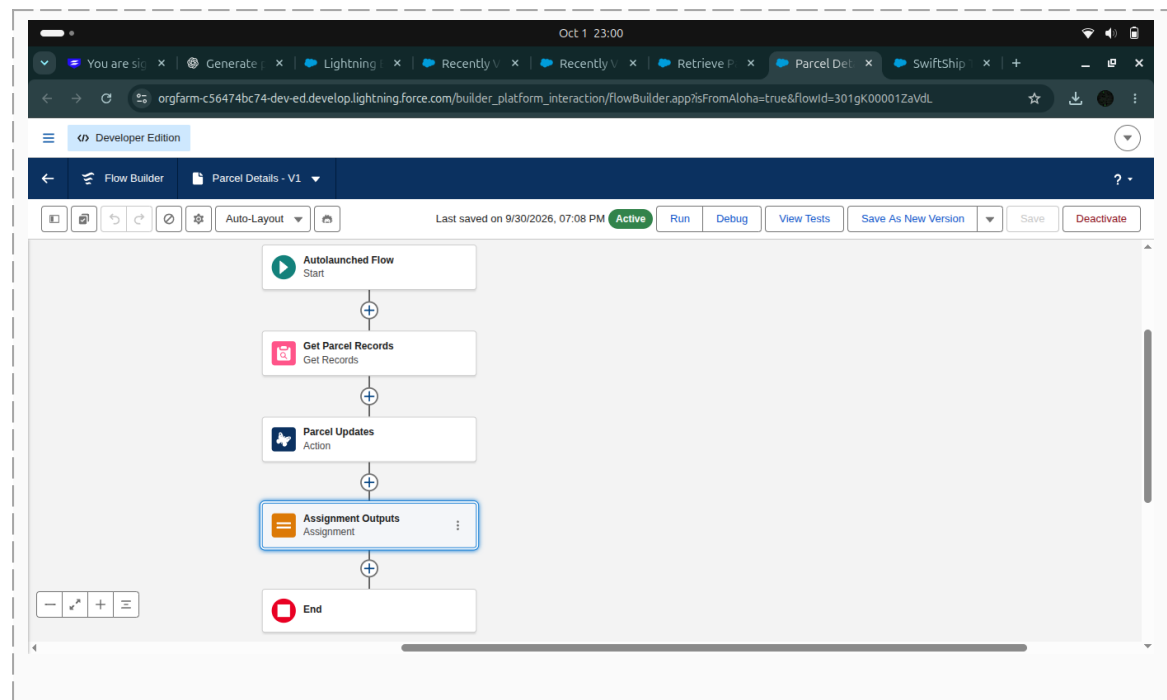


Figure 12.46: Full Flow

13. AGENTFORCE AI WORKING

Agentforce provides the conversational entry point for parcel tracking.

The AI path is controlled through the configured Flow and Prompt Builder action rather than relying only on free-form generation.

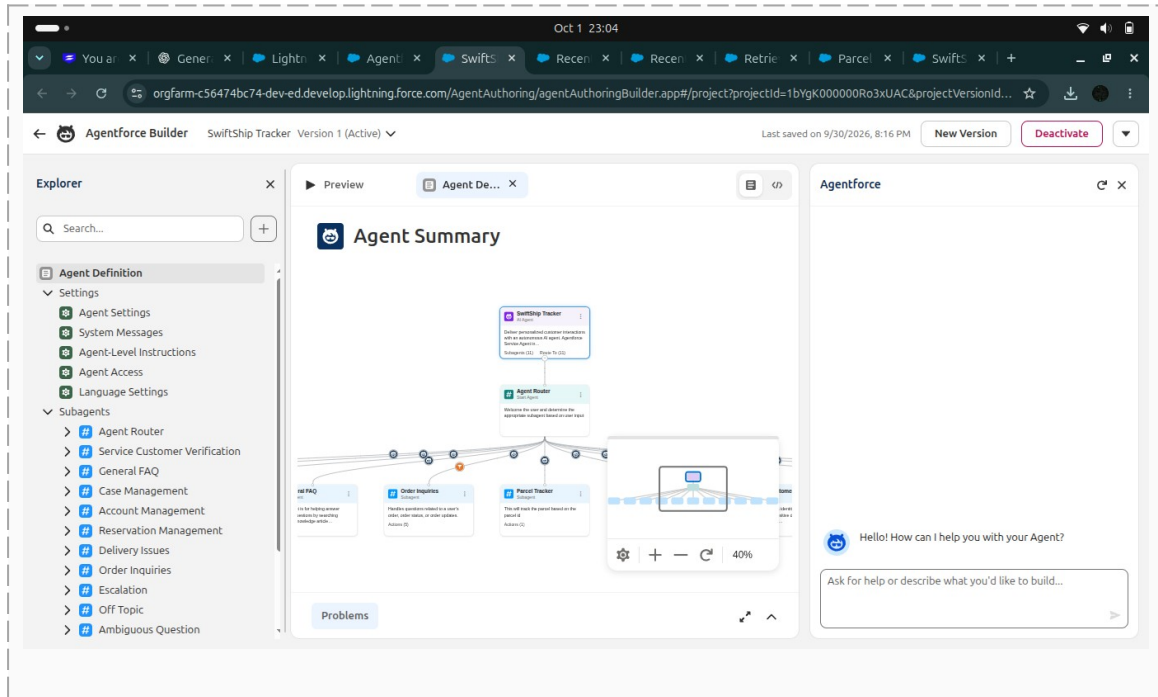


Figure 13.47: Agentforce Builder

13.1 End-to-End AI Lifecycle

- User asks for parcel information.
- Agent action invokes Parcel_Details.
- Ids receives the parcel identifier.
- Get_Parcel_Records retrieves the matching Parcel__c record.
- Retrieve_Parcel_Details generates the response.
- Assignment_Outputs places the response in Output.
- The response is returned to the conversational interface.

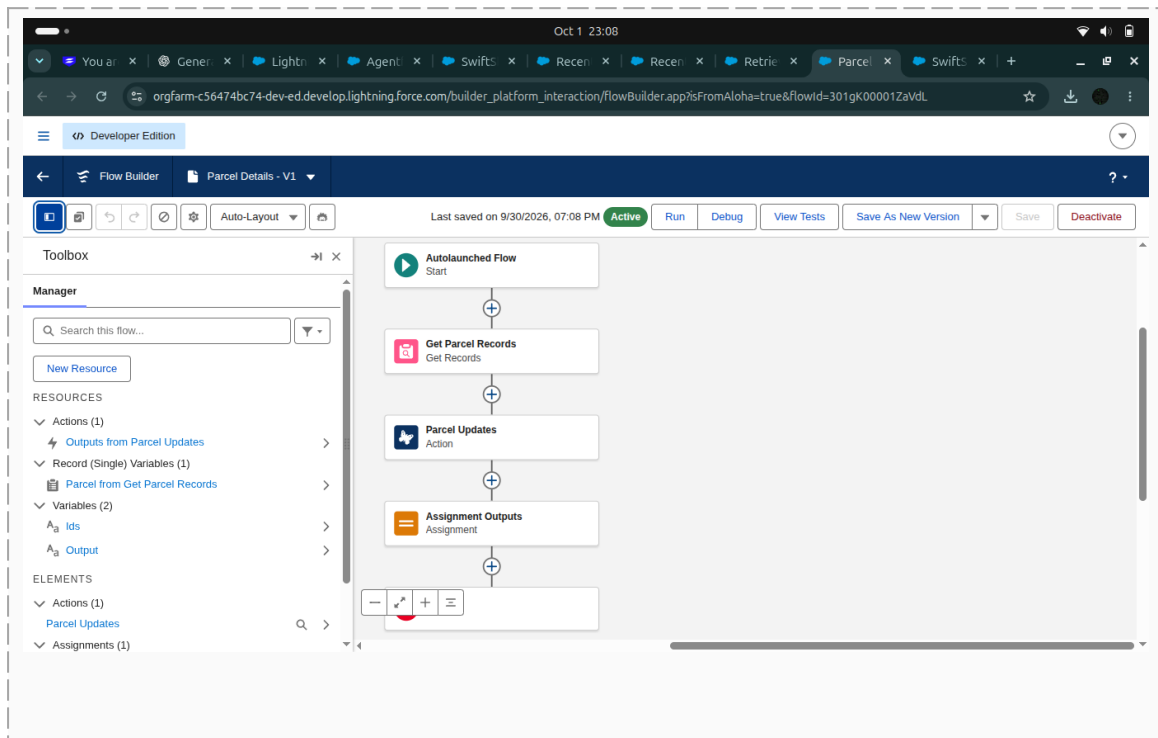


Figure 13.48: AI Lifecycle

13.2 Why the AI Agent Matters

The AI component demonstrates how conversational requests can be connected to structured Salesforce records.

It is one of the strongest project demonstration areas because it connects the user experience, Flow, data model and Prompt Builder.

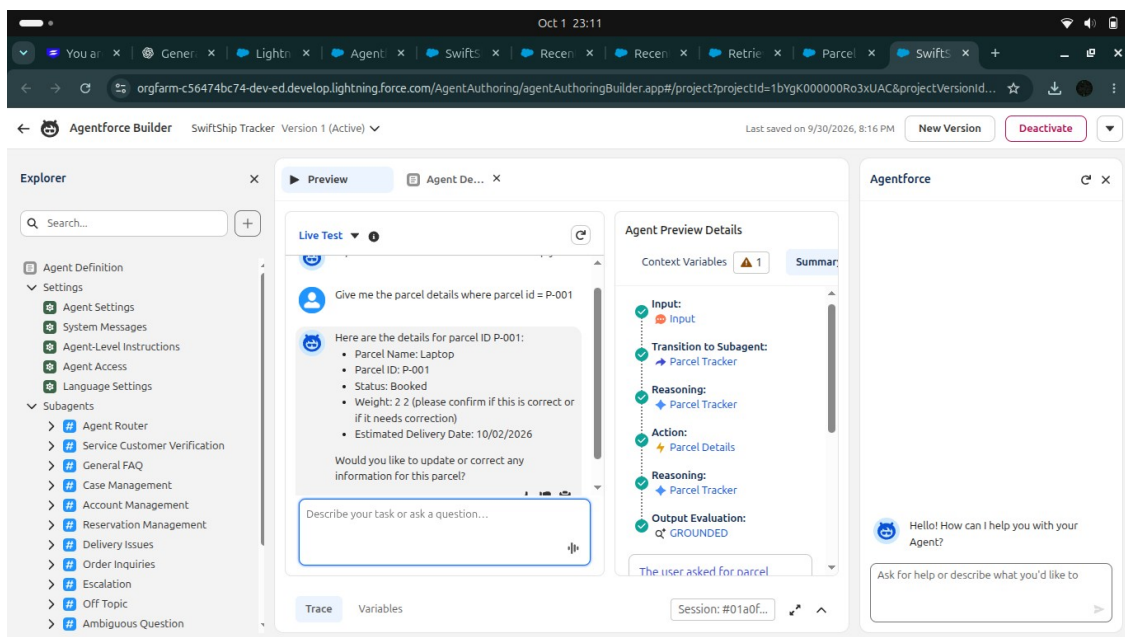


Figure 13.49: Agentforce Conversation

14. TESTING STRATEGY

Testing covers object configuration, Flow behavior, AI response generation, permissions, CLI validation and source control.

The successful CLI dry run validated 32 components.

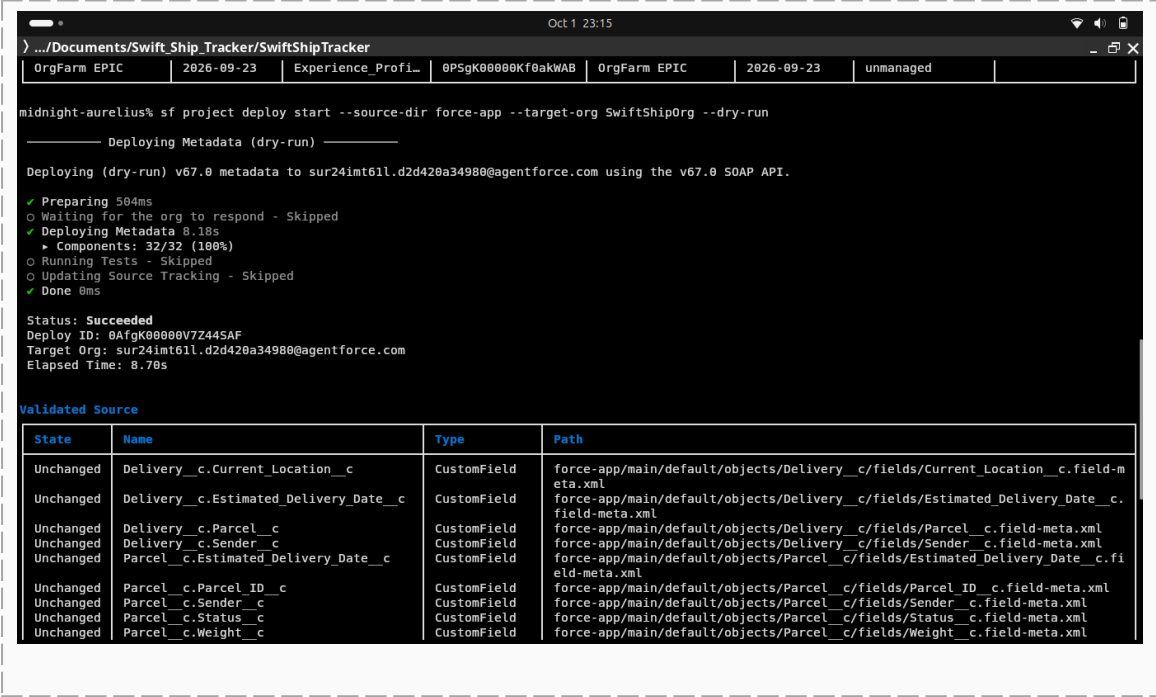


Figure 14.50: Testing Overview

14.1 Functional Test Cases

Table 14.11: Functional Test Cases

ID	Test	Expected Result
TC-01	Open Parcel	Available
TC-02	Open Delivery	Available
TC-03	Open Sender	Available
TC-04	Open Receiver	Available
TC-05	Valid Parcel ID	Record retrieved
TC-06	Run Flow	Output returned
TC-07	Prompt action	Response generated
TC-08	Permission set	Permissions available
TC-09	CLI dry run	Succeeded

ID	Test	Expected Result
TC-10	Git status	Clean / synchronized

14.2 Flow Testing

Use a Parcel_ID__c value that exists in the org. Confirm that Get_Parcel_Records finds the record and that the Flow completes with an Output value.

Also test an invalid identifier to document behavior when no matching record exists.

14.3 AI Testing

Test with a known parcel identifier.

Verify that the returned Parcel ID, Status, Weight and Estimated Delivery Date correspond to Salesforce data.

15. SALESFORCE CLI VALIDATION

Salesforce CLI was used for authentication, metadata listing, data query, deployment validation and source inspection.

An initial dry-run returned a fetch failed error, but the verbose execution subsequently completed successfully.

15.1 Successful Dry Run

Command: sf project deploy start --source-dir force-app/main/default --target-org SwiftShipOrg --dry-run --verbose. Result: Status Succeeded. The deployment validated 32 components and skipped test execution.

15.2 Metadata Validation Summary

Table 15.12: Metadata Validation Summary

Metadata	Validated
Custom Fields	17
Custom Objects	4
Custom Tabs	4
Flow	1
List Views	4
Permission Sets	2

Metadata	Validated
Total	32

16. GITHUB INTEGRATION

The Salesforce DX project was connected to the SwiftShipTracker GitHub repository.

The origin remote points to the repository and the main branch was pushed successfully.

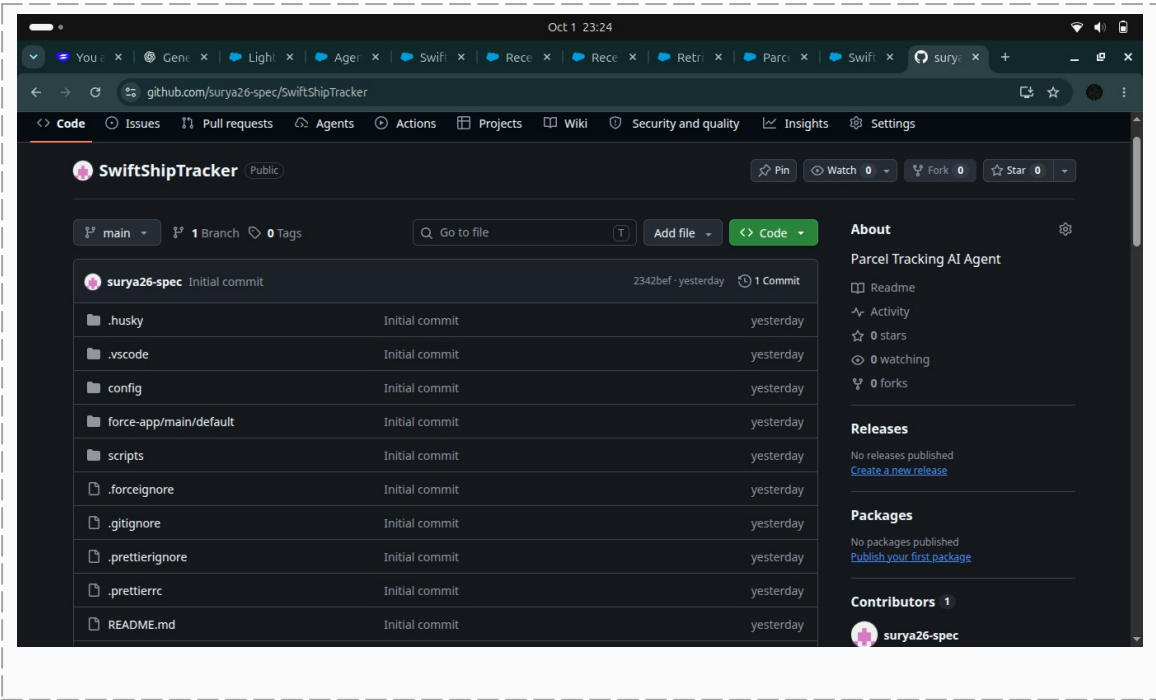


Figure 16.51: GitHub Repository

16.1 Git Status

The final status showed the main branch was up to date with origin/main and the working tree was clean.

This confirms source synchronization at the time of validation.

16.2 Git Workflow

- Modify Salesforce metadata.
- Review git status.
- Commit changes.
- Push to origin/main.
- Verify synchronization.

- Use GitHub history for traceability.

17. SOURCE CODE ORGANIZATION

Objects are stored under force-app/main/default/objects.

Flows are stored under force-app/main/default/flows.

Permission sets are stored under force-app/main/default/permissionsets.

Tabs are stored under force-app/main/default/tabs.

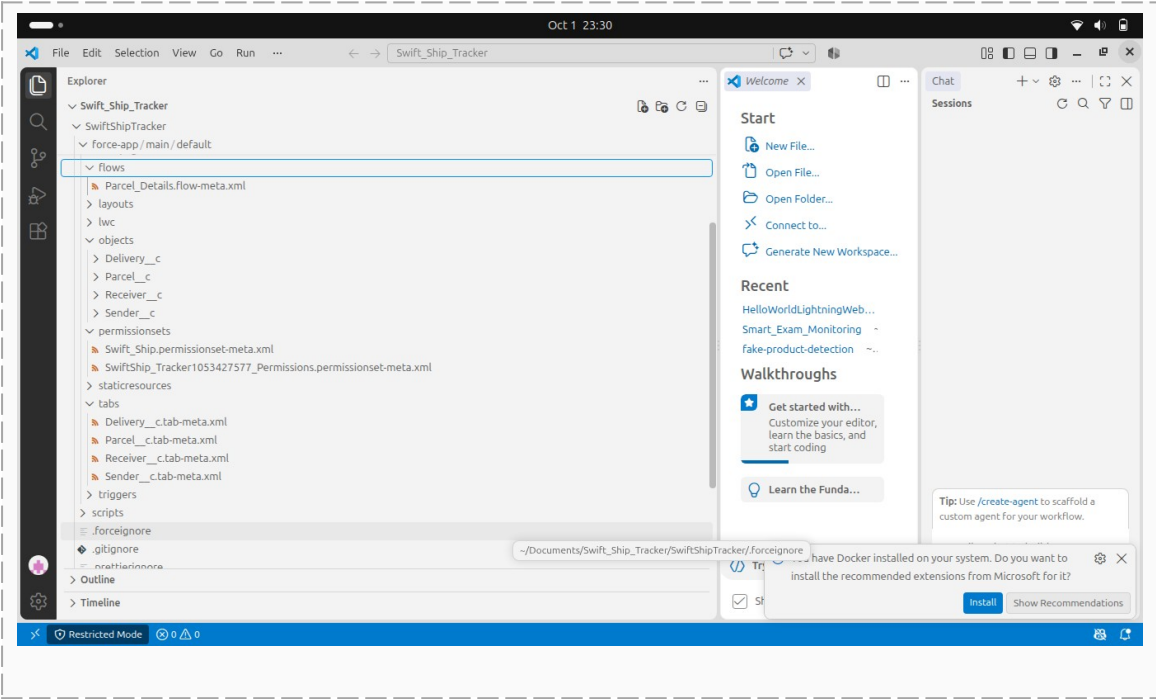


Figure 17.52: VS Code Metadata Tree

17.1 Important Files

Table 17.13: Important Files

sfdx-project.json	Salesforce DX configuration
objects/	Custom object metadata
flows/	Flow metadata
permissionsets/	Permission set metadata
tabs/	Custom tab metadata
README.md	Project documentation
package.json	Project package configuration

18. DEPLOYMENT CONSIDERATIONS

The target org did not support the default source-tracking retrieve command.

Metadata listing was used to verify org components, and the source was successfully validated with a verbose dry-run deployment.

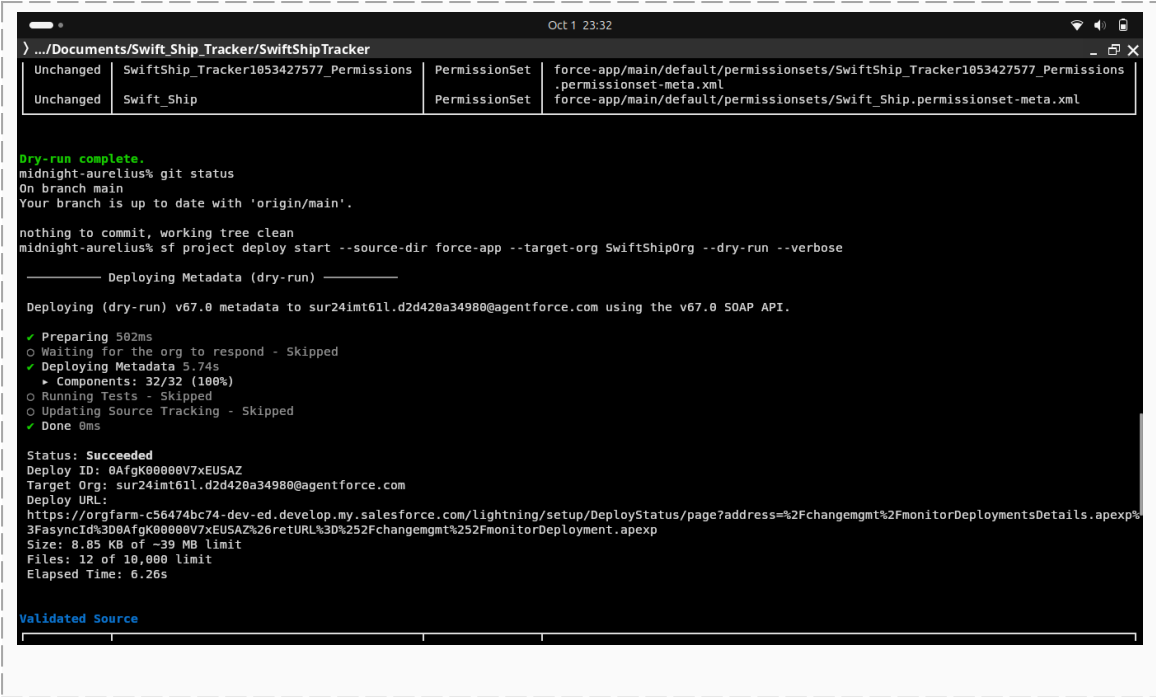


Figure 18.53: Deployment Troubleshooting

18.1 Troubleshooting History

Table 18.14: Troubleshooting History

Issue	Observation	Outcome
Authentication	AuthCodeExchangeError initially	Later web login succeeded
Retrieve	noSourceTracking	Use explicit metadata
Flow retrieve	fetch failed	Metadata listing succeeded
Deploy dry run	fetch failed initially	Verbose run succeeded
Git remote	origin already existed	Remote verified

19. PROJECT LIMITATIONS

- Developer Org implementation only.
- No SwiftShip Apex classes were found.
- No Reports or Dashboards were found in the target org.
- No production or sandbox deployment was performed.
- No external logistics integration is implemented.
- Advanced real-time GPS tracking is not claimed by the current source.
- AI output depends on correct data and configuration.

20. ADVANTAGES

- Centralized parcel data.
- Separate business entities.
- Simple Parcel ID retrieval.
- AI-assisted response.
- Reusable Flow.
- Permission-based access.
- Salesforce DX maintainability.
- GitHub source history.
- CLI deployment validation.

20.1 Disadvantages

- Depends on Salesforce configuration.
- AI depends on Prompt Builder configuration.
- Missing data can produce incomplete responses.
- Analytics are limited without reports and dashboards.
- External integrations are outside the current scope.
- Production release requires additional testing and governance.

21. FUTURE SCOPE

- Reports and dashboards.
- Additional Agentforce topics and actions.
- More Prompt Builder templates.
- Advanced delivery tracking.
- External logistics integrations.
- Experience Cloud customer portal.

- Automated notifications.
- Sandbox and CI/CD deployment.
- Improved error handling.

22. CONCLUSION

SwiftShip Tracker demonstrates a practical Salesforce parcel-management solution using custom objects, Flow automation, Prompt Builder, Agentforce integration, permission sets, Salesforce DX and GitHub.

The central technical flow is Parcel ID → Salesforce record retrieval → Prompt Builder → Flow Output → conversational response.

23. PROJECT DEMONSTRATION SEQUENCE

- Show the four objects.
- Show Parcel fields.
- Show relationships.
- Show tabs and list views.
- Show permission sets.
- Show Prompt Builder.
- Show Parcel_Details Flow.
- Show Agentforce preview.
- Show successful CLI dry run.
- Show GitHub repository.

24. APPENDIX A — CLI COMMANDS

```
sf --version
sf org login web --alias SwiftShipOrg
sf org list
sf org display --target-org SwiftShipOrg
sf data query --target-org SwiftShipOrg --query "SELECT Id, Name FROM Organization
LIMIT 1"
sf org list metadata --metadata-type Flow --target-org SwiftShipOrg
sf org list metadata --metadata-type PermissionSet --target-org SwiftShipOrg
```

```
sf project deploy start --source-dir force-app/main/default --target-org SwiftShipOrg --dry-run --verbose
```

25. APPENDIX B — METADATA INVENTORY

Table 25.15: Appendix B — Metadata Inventory

Category	Actual Project Content
Objects	Delivery__c, Parcel__c, Receiver__c, Sender__c
Flow	Parcel_Details
Permission Sets	Swift_Ship; SwiftShip_Tracker1053427577_Permissions
Tabs	Delivery__c; Parcel__c; Receiver__c; Sender__c
List Views	All view for each custom object

26. APPENDIX C — SCREENSHOT CHECKLIST

- Object Manager — Parcel
- Parcel fields
- Delivery fields
- Sender fields
- Receiver fields
- Relationships
- Custom tabs
- Permission sets
- Prompt Builder
- Prompt resources
- Flow canvas
- Get Records
- Prompt Action
- Assignment
- Agentforce Builder
- Agentforce chat
- CLI dry-run
- Git status
- GitHub repository

27. APPENDIX D — SCREENSHOT PLACEMENT GUIDE

Use one clear screenshot for each major configuration. Avoid repeated screenshots that prove the same thing.

Place configuration screenshots immediately after the section that explains them.

For AI, include both configuration and a successful conversational result.

For deployment, include the complete successful verbose dry-run output.

28. APPENDIX E — FINAL EVIDENCE

The strongest implementation evidence is the successful dry-run showing 32 validated components.

Additional evidence comes from the Salesforce metadata listings, Flow definition, Agentforce interaction and GitHub synchronization.

29. FINAL REVIEW CHECKLIST

- All four objects visible.
- Parcel ID field visible.
- Flow Active.
- Prompt Builder action available.
- Permission sets present.
- GitHub repository synchronized.
- CLI dry run succeeded.
- Screenshots readable.
- Implemented features clearly separated from future scope.

30. FINAL SUBMISSION NOTES

Replace every screenshot placeholder with the corresponding Salesforce, terminal, VS Code or GitHub screenshot.

Keep screenshots readable and crop unnecessary browser areas.

Do not claim Apex, dashboards, Experience Cloud or external integrations as completed unless they are actually added.

The structure follows the detailed milestone-oriented style of the 67-page reference PDF while remaining faithful to the implemented SwiftShip Tracker project.